Running Jobs:
Part 1: Batch Computing
Part 2: Interactive Computing

COMPLECS Team

https://bit.ly/COMPLECS
https://github.com/sdsc-complecs

SDSC
SAN DIEGO SUPERCOMPUTER CENTER
UC San Diego

COMPLECS: Batch Computing Series

Working with the Linux Scheduler

How to interact with the Linux scheduler and run your research workloads on your personal computer, a shared workstation, or even a high-performance computing system.

Getting Started with Batch Job Scheduling: Slurm Edition

How to schedule your batch jobs on high-performance computing systems using the Slurm Workload Manager.

High-Throughput and Many-Task Computing Workflows: Slurm Edition

How to build and run your high-throughput and many-task computing workflows on high-performance computing systems using the Slurm Workload Manager.

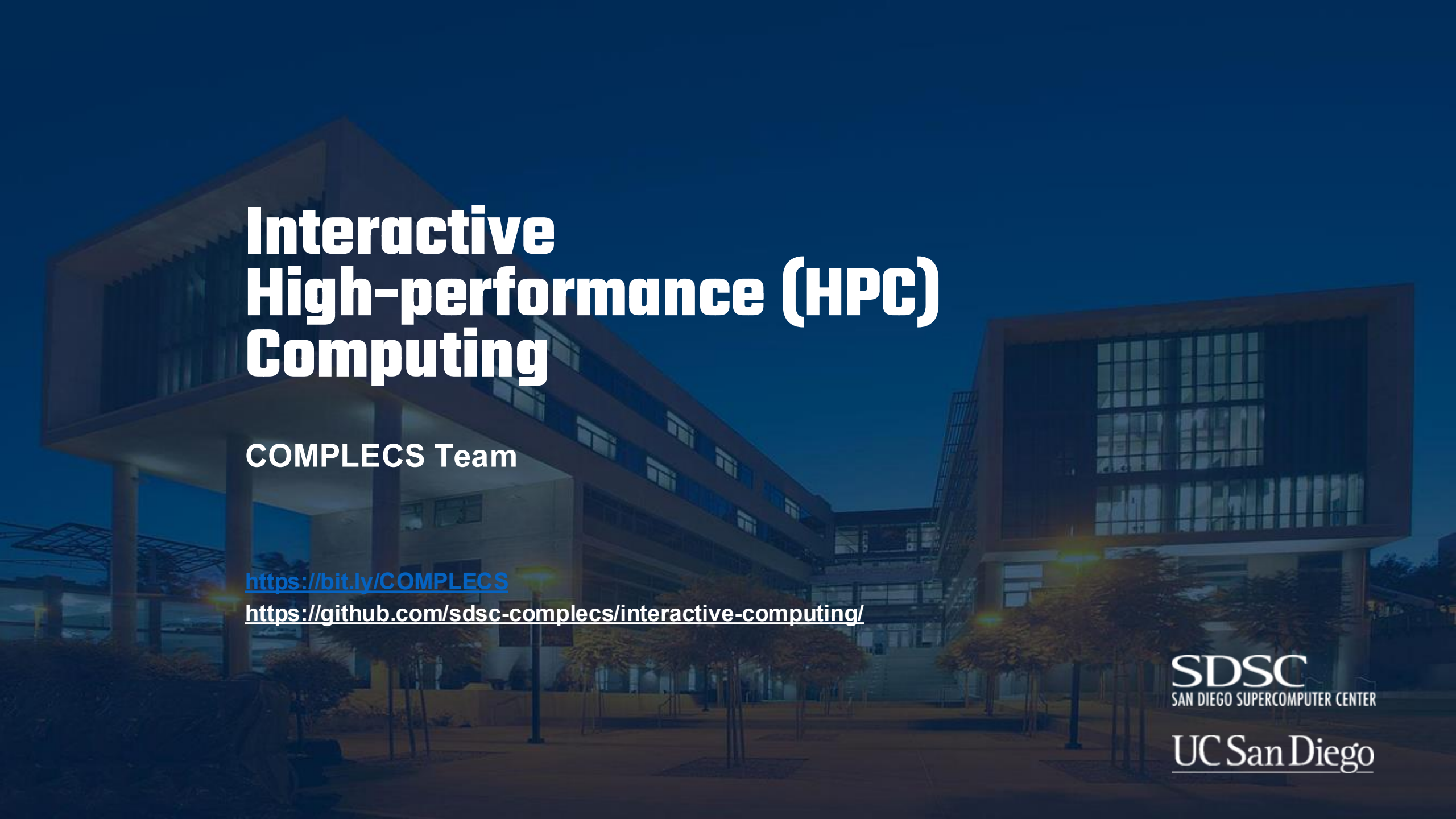
Getting Started with Batch Job Scheduling: Slurm Edition

Learning Goals:

- What is a batch job?
- What is Slurm?
- How to start interacting with Slurm
- How to write, submit and run your first batch job on a high-performance computing system

Prerequisite Knowledge:

- Parallel Computing Concepts
- Intermediate Linux and Shell Scripting



Interactive High-performance (HPC) Computing

COMPLECS Team

https://bit.ly/COMPLECS
https://github.com/sdsc-complecs/interactive-computing/

SDSC
SAN DIEGO SUPERCOMPUTER CENTER

UC San Diego

Table of Contents: Part 1

HPC Compute Node: Dell C4140

Figure 1. Front view of the system

1. Control panel

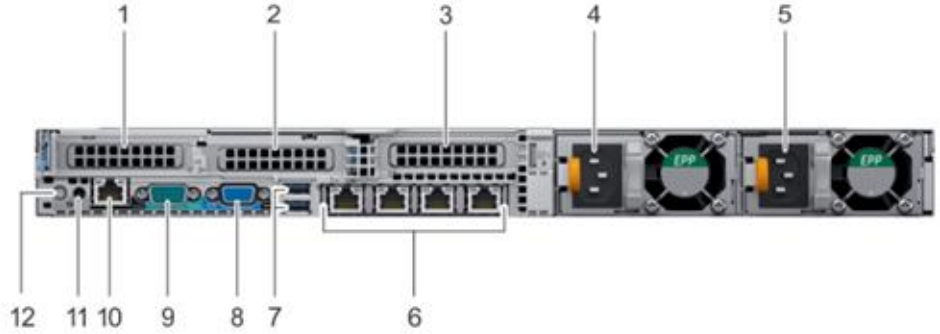


Figure 2. Rear view of the system

1. PCIe expansion card slot 1
2. PCIe expansion card slot 2

   NOTE: This slot is dedicated for BOSS c
3. PCIe expansion card slot 3
4. Power supply unit (PSU 1)
5. Power supply unit (PSU 2)
6. Ethernet connectors (4)
7. USB connectors (2)
8. Video connector
9. Serial connector
10. iDRAC Enterprise port
11. NMI button
12. System identification button

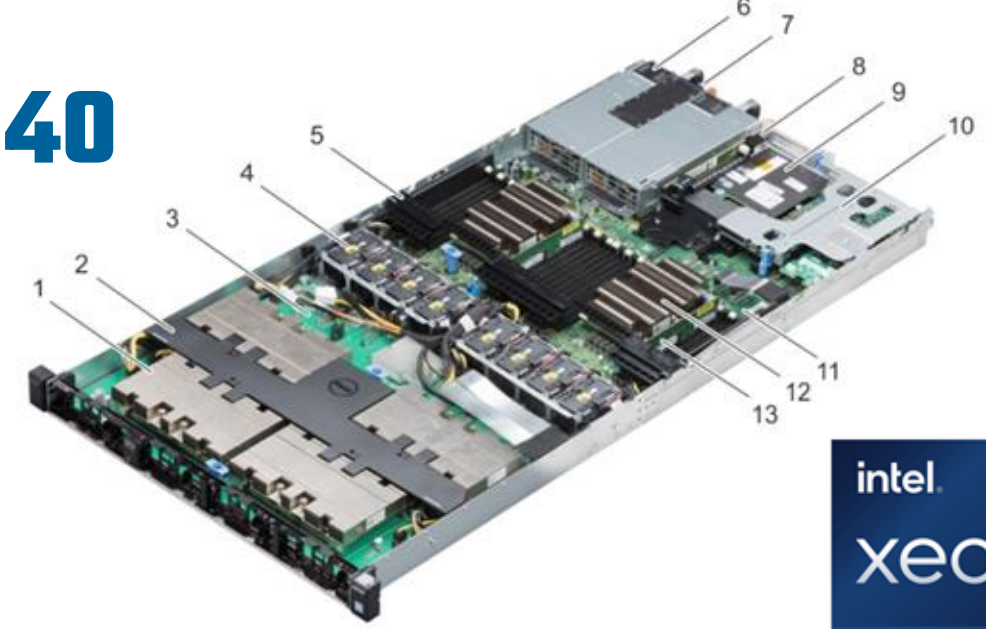


Figure 4. Configuration K

1. NVLink Heat sink and processor (4)
2. NVLink air shroud
3. NVLink board
4. Cooling fan (8)
5. Air shroud
6. PSU (2)
7. Information tag
8. Riser 2A (Low-profile PCIe expansion card: Slot 3)
9. Network daughter card (NDC)
10. Riser 1A (Low-profile PCIe expansion cards: Slots 1 and 2)
11. System board
12. Processor and heat sink (2)
13. DIMMs (24)

[NVIDIA V100 32GB SMX2]

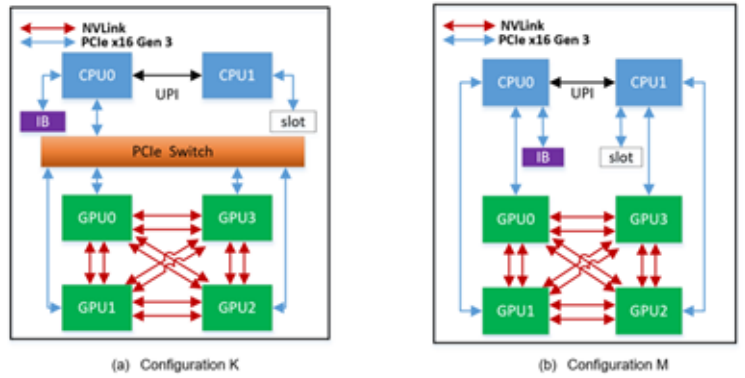
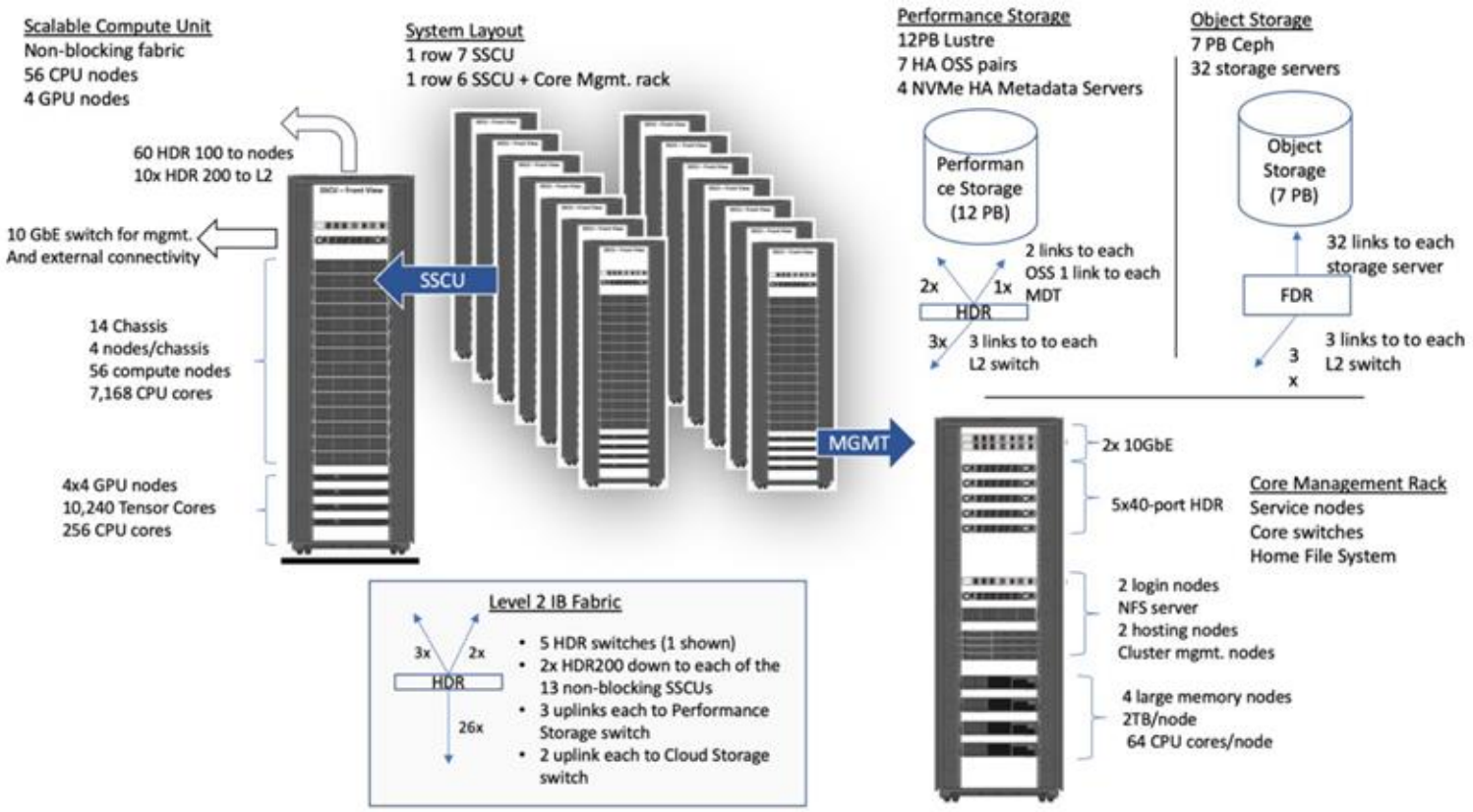


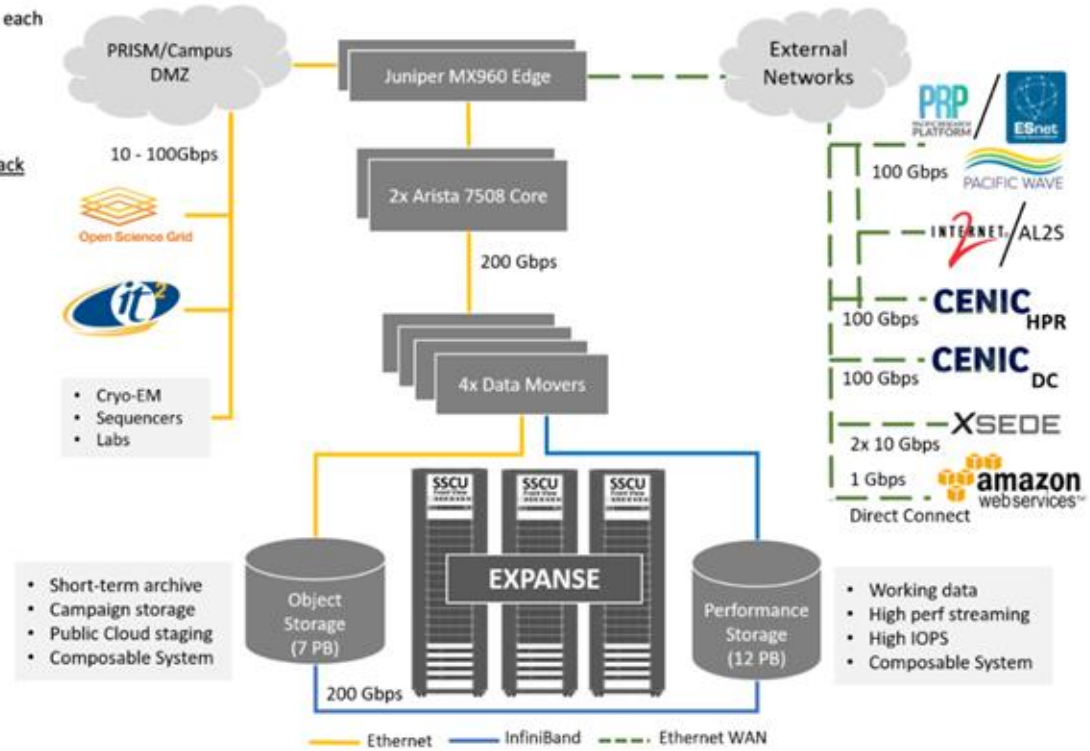
Figure 1: The comparison between configuration K and configuration M in C4140 server

intel. XEON GOLD

[Intel Xeon Gold 6248]

HPC System Architecture: Expanse @ SDSC





https://access-ci.org

https://expanse.sdsc.edu

https://www.youtube.com/watch?v=uNZyg6X_t3s

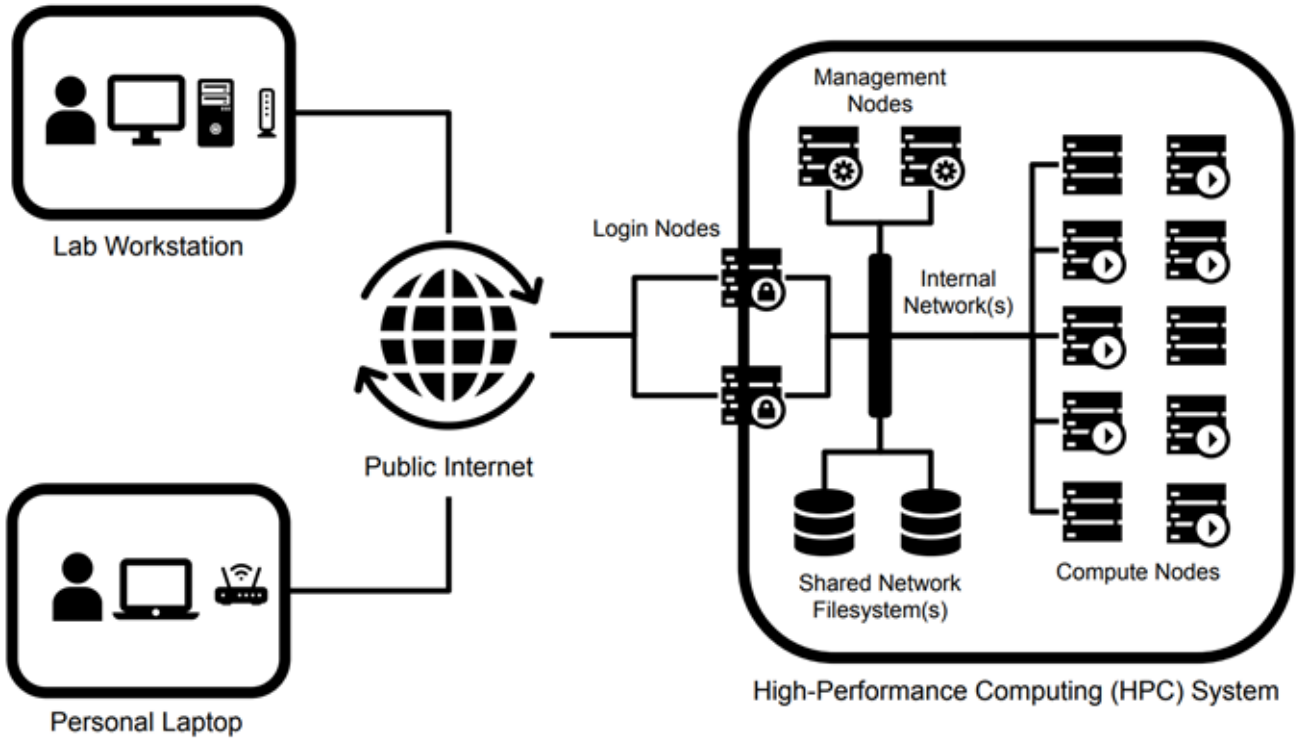
HPC System Architecture: Conceptual Model

Login node(s): Provide remote access to an HPC system; use only for simple tasks such as editing files, limited data transfers to and from the system, and batch job submission

Compute nodes: Run computational workloads: simulations, data analysis and visualization

Internal Network(s): Provide high-bandwidth, low-latency communication between computes nodes ; access to shared (parallel) filesystems; system management

Shared Network Filesystem(s): Provide input/output (I/O) access to data storage systems from any compute node



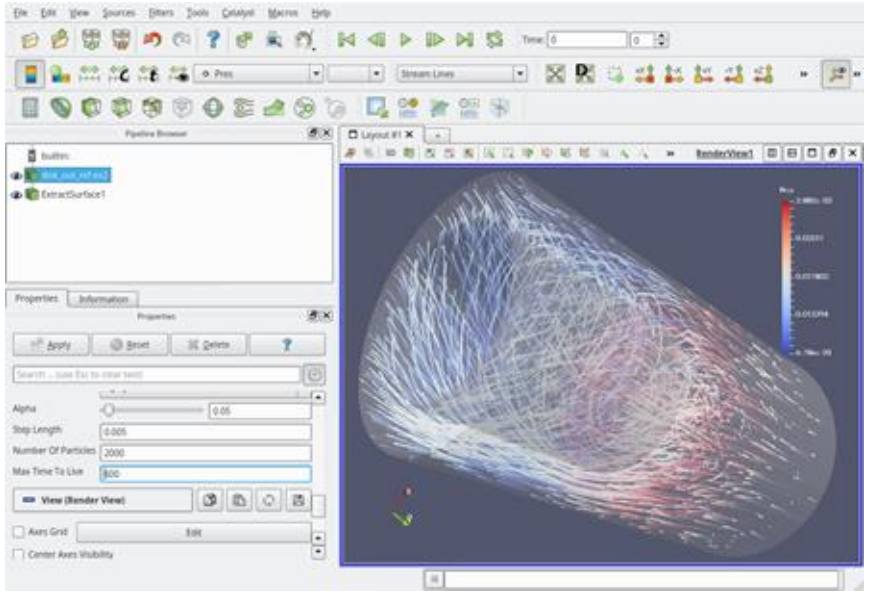
Management node(s): Run core system services such as cluster management software, system monitoring software, *batch job scheduler*, etc

Table of Contents: Part 1

- **High-Performance Computing Hardware and System Architecture**
- **Batch Job Scheduling and Resource Management**
- **Getting Started with the Slurm Workload Manager**
  - **`squeue`** - view information about ***jobs*** located in the Slurm scheduling ***queue***
  - **`sinfo`** - view information about Slurm ***nodes*** and ***partitions***
- **First Batch Job(s) with Slurm**
  - **`sbatch`** - submit a ***batch script*** to Slurm
  - **`scancel`** - used to signal or cancel jobs, job arrays or job steps
  - **`sacct`** - displays accounting data for all jobs in the Slurm database
- **Best Practices with Slurm**

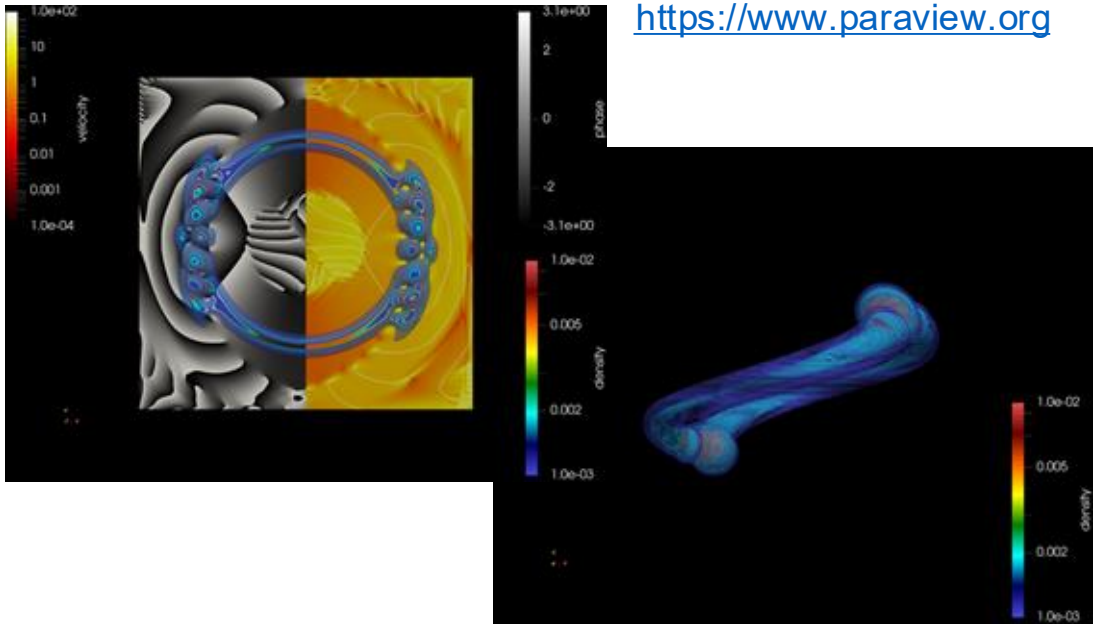
Interactive vs. Batch Computing

Interactive computing is the use of computer programs that accept input from the user as they execute on a system. Examples in HPC include the use of software applications for code development and performance optimization as well as real-time data exploration, analysis and visualization.



https://www.paraview.org

Batch processesing involves the execution of computer programs in an automated and unattended way on a system without user interaction. Batch computing systems are widely used to automate high-volume, repetitive tasks such as data processing and to ***allow the shared use of advanced compute resources like an HPC system*** among many users.

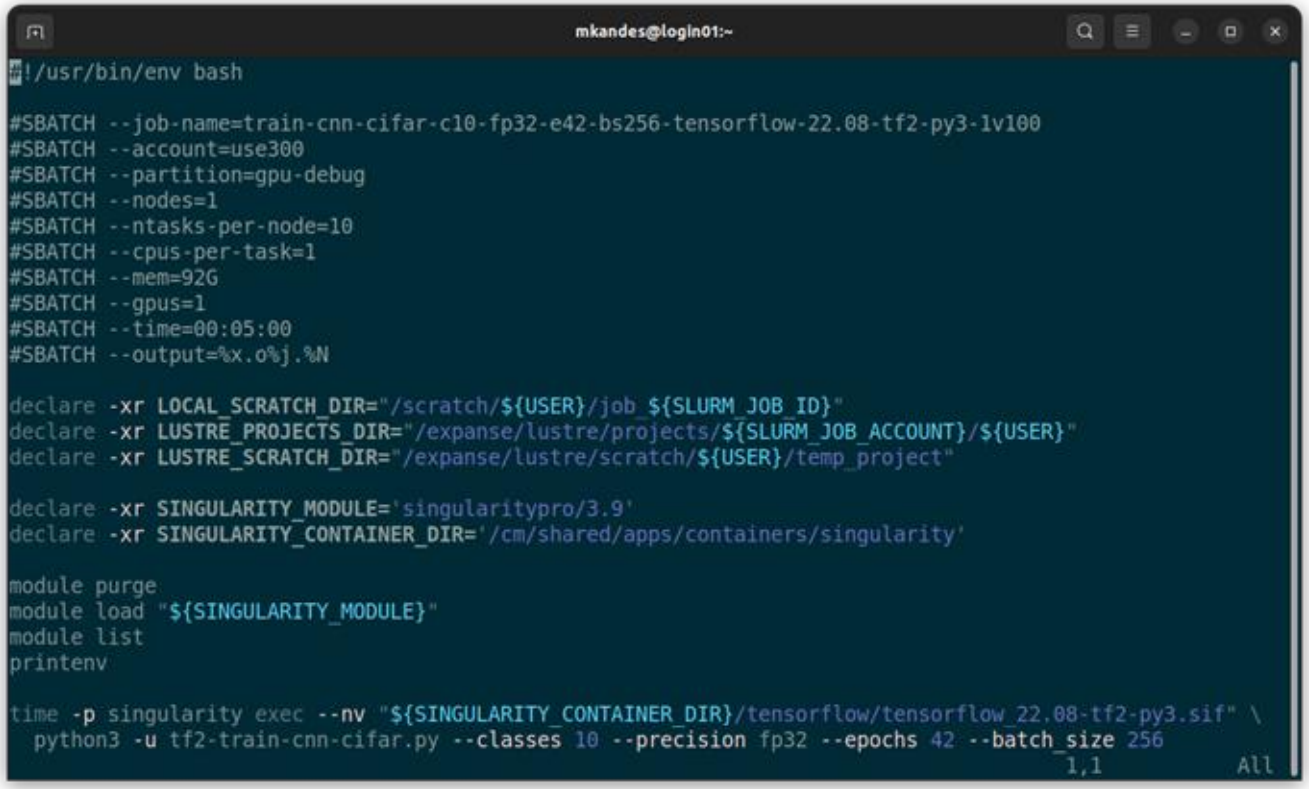


What is a Batch Job?

A **batch** job is a pre-*scripted* sets of commands that are executed as processes, which may or may not be multi-threaded, on a certain type or set of dedicated compute resources within a computing system for a given amount of time without user interaction.

On HPC systems, batch jobs may run large-scale simulations, perform complex data analysis, or any other types of demanding computational tasks.

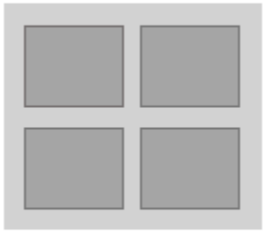
A batch job is specified as a standard shell script that is then submitted to a queue of other batch jobs managed by a scheduler, which assigns the job to available resources and ensures that the job runs to completion.



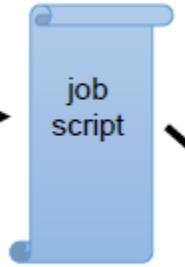
What is a Batch Job Scheduler?

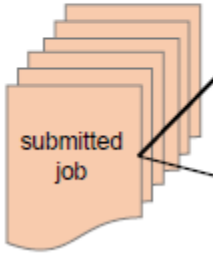
A **batch job scheduler** is software that controls and tracks *where* and *when* batch jobs submitted to a computing system will eventually run on its compute resources.

user

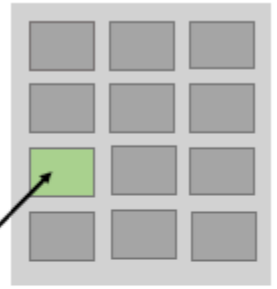


login / front-end nodes

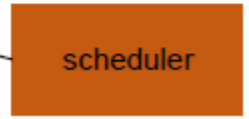




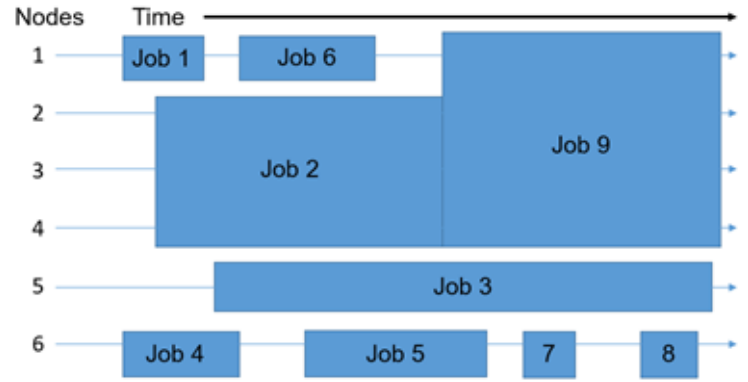
job queue



batch system



Batch Job Scheduling in Practice



Batch job schedulers typically have three primary aims:

- Minimize the time between the job submission and job completion: no user job should stay pending in the queue for too long
- Optimize compute resource utilization: no compute resource should be idle for too long
- Maximize job throughput: process as many jobs as soon as possible

In most cases, the batch job schedulers on HPC systems will employ some form of a fairshare scheduling algorithm to match user jobs to compute nodes over time.

** All interactive jobs you run on an HPC system are usually treated the same as batch jobs.*

Table of Contents: Part 1

- **High-Performance Computing Hardware and System Architecture**
- **Batch Job Scheduling and Resource Management**
- **Getting Started with the Slurm Workload Manager**
  - **`squeue`** - view information about ***jobs*** located in the Slurm scheduling ***queue***
  - **`sinfo`** - view information about Slurm ***nodes*** and ***partitions***
- **First Batch Job(s) with Slurm**
  - **`sbatch`** - submit a ***batch script*** to Slurm
  - **`scancel`** - used to signal or cancel jobs, job arrays or job steps
  - **`sacct`** - displays accounting data for all jobs in the Slurm database
- **Best Practices with Slurm**

Slurm Workload Manager

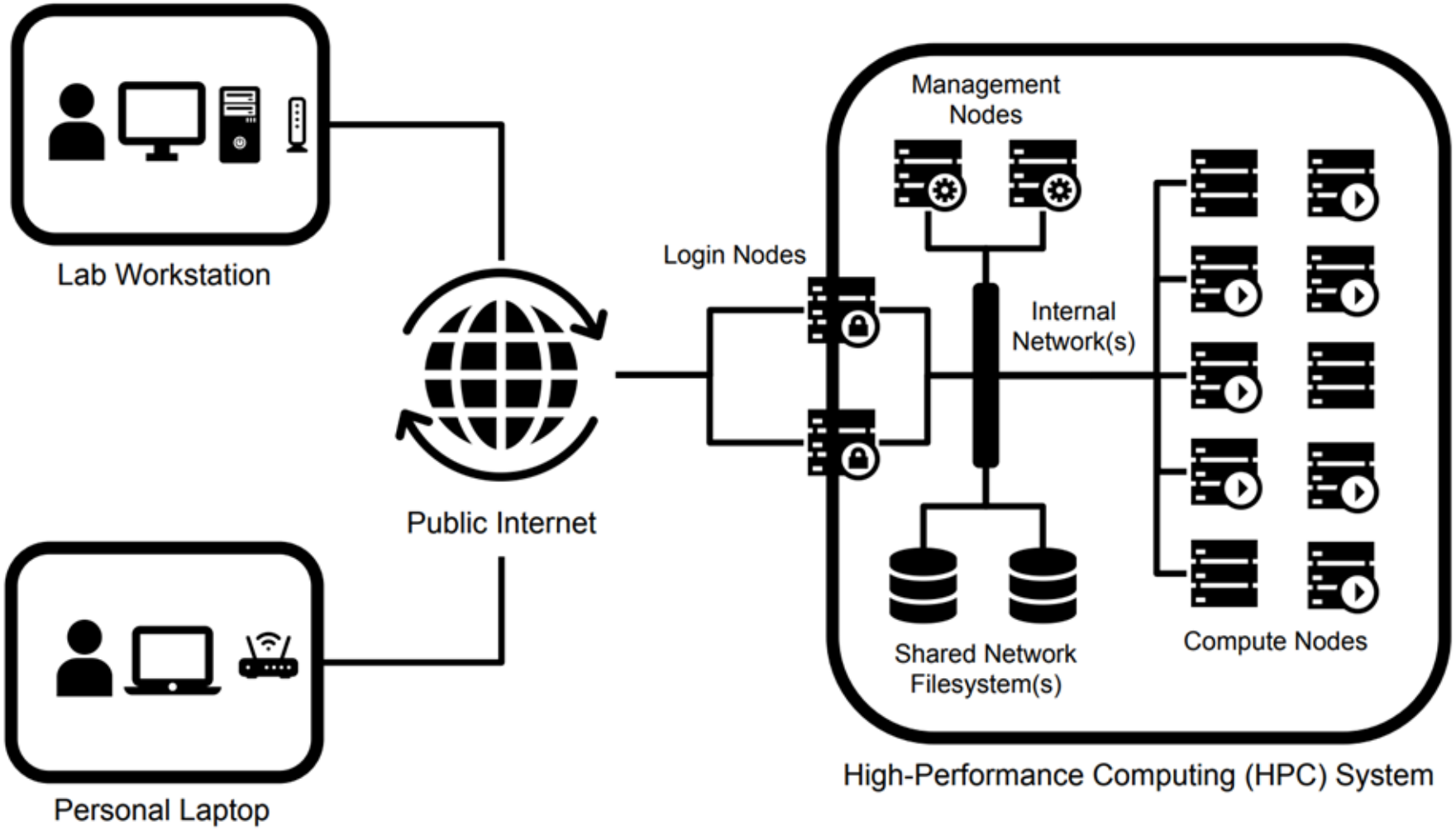
The Slurm Workload Manager is a free and open-source distributed batch job scheduler and resource manager for Unix-like operating systems used by the majority of the world's supercomputers and computer clusters today.

Slurm provides three key functions:

- the allocation of compute nodes for some period of time to process user jobs;
- a framework for starting, executing, and monitoring the progress of user jobs across a set of allocated compute nodes; and
- arbitrating the demand for compute nodes by managing a queue of pending user jobs

For more information, see the Slurm documentation: https://slurm.schedmd.com

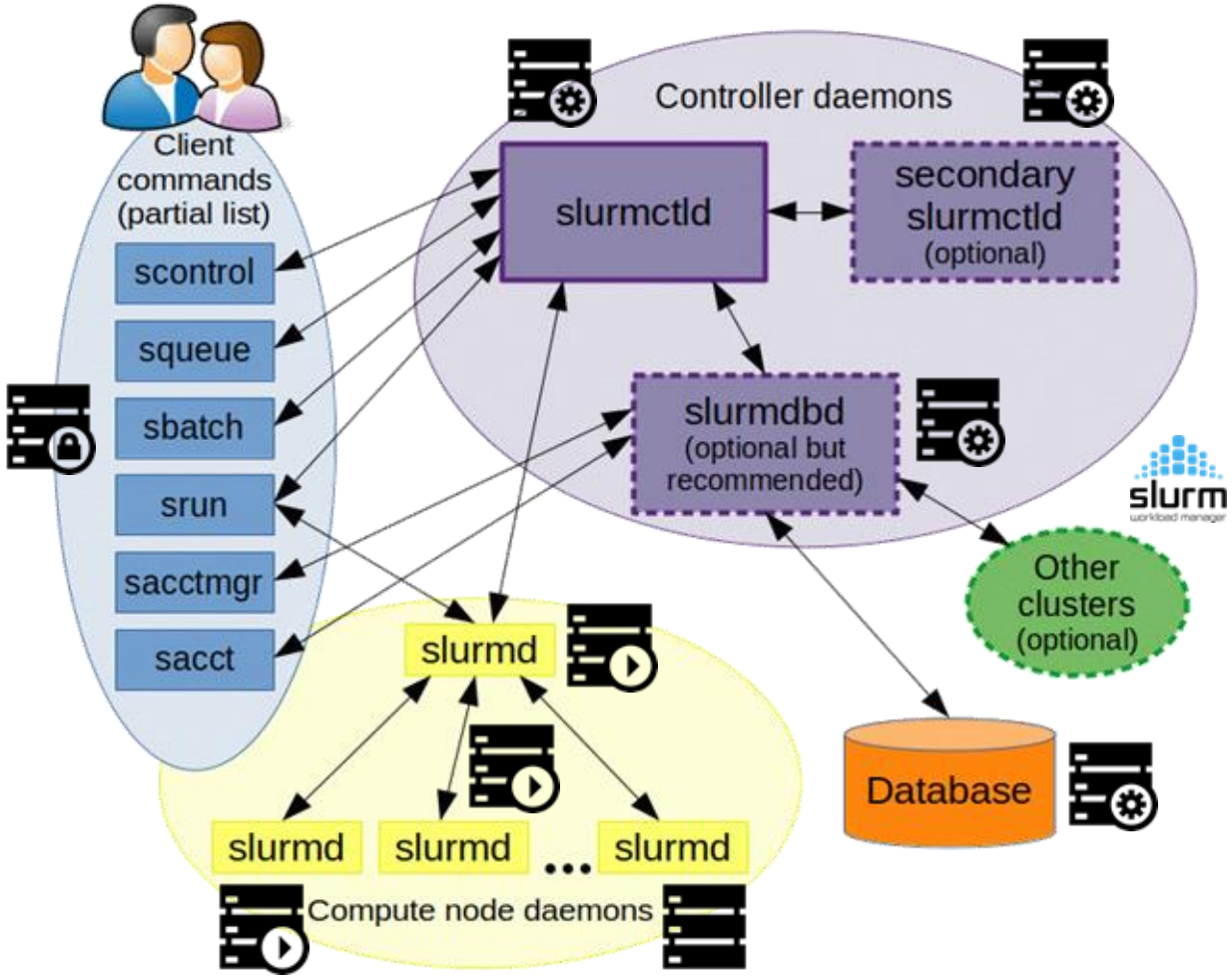
HPC System Architecture: Conceptual Model



Slurm Architecture

Slurm is comprised of the three following types of service daemons:

- **`slurmd`** is the compute node daemon; it monitors all jobs running on the compute node, accepts jobs, launches jobs, and stops running jobs
- **`slurmctld`** is the central management daemon; it monitors all other Slurm daemons and resources, accepts jobs, and allocates resources to the jobs.
- **`slurmdbd`** is the accounting database daemon; it provides an interface for archiving accounting records to an external database



https://slurm.schedmd.com/quickstart.html

Slurm User Commands

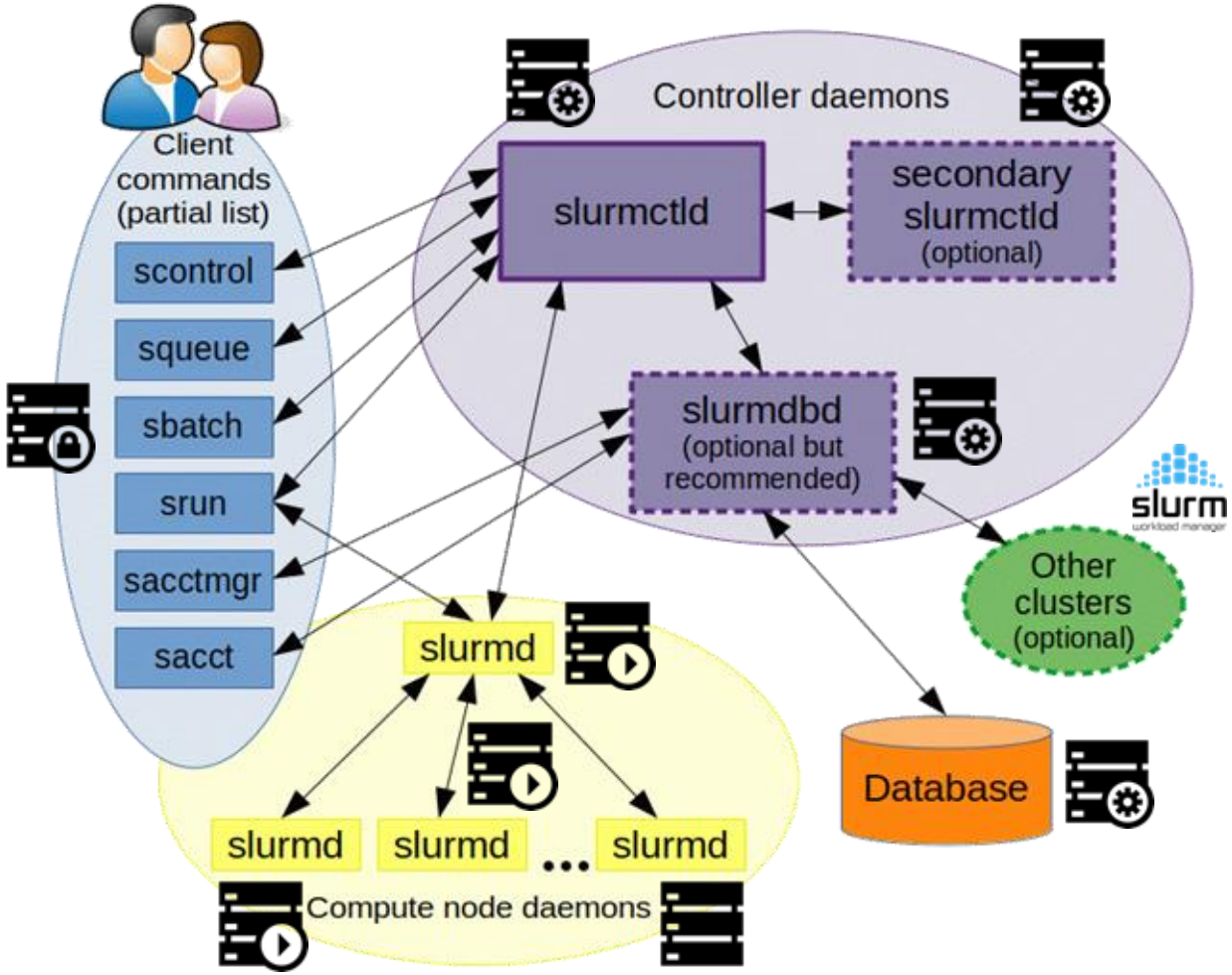
The essential client commands you'll want to learn how to use as you get started with Slurm are as follows:

- **sbatch**
- **squeue**
- **scancel**

Other client commands you may find useful from time to time are:

- **sacct**
- **sinfo**

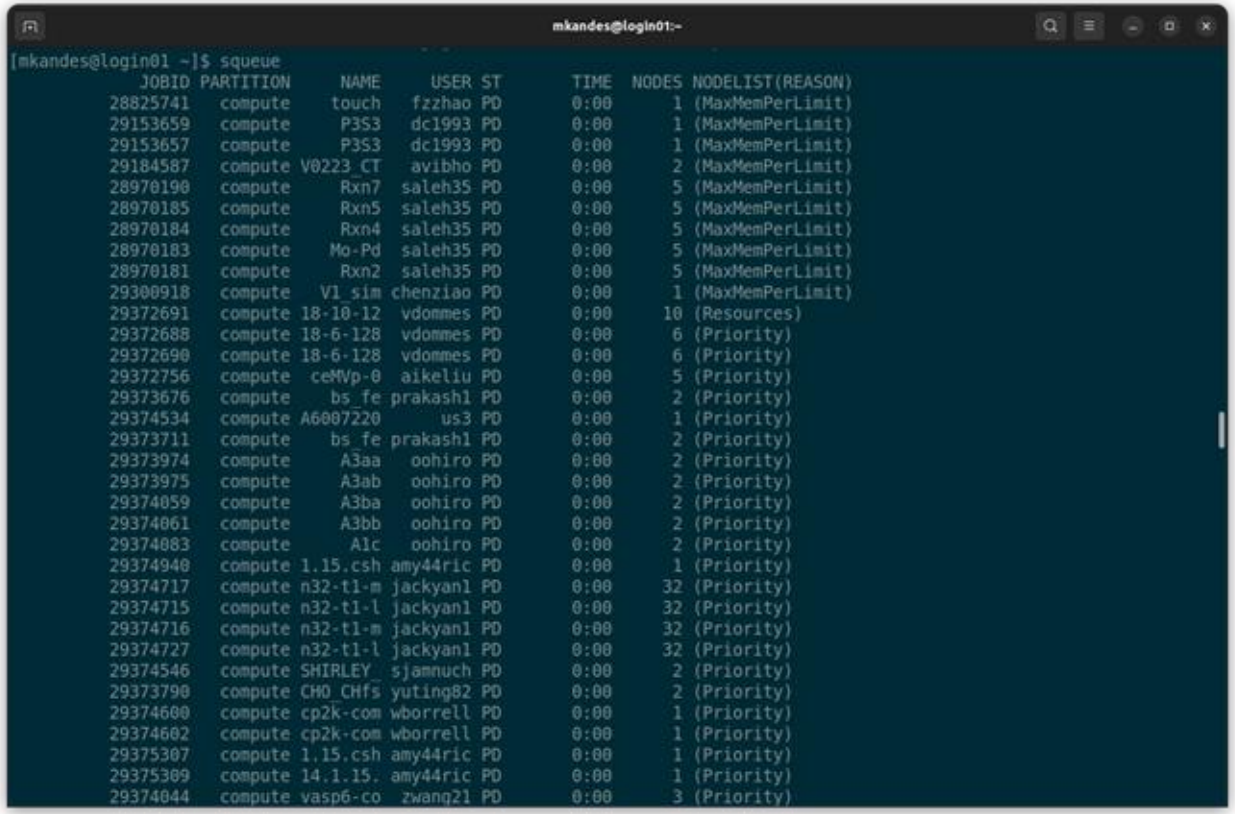
You will generally only run these commands interactively on a system's login nodes.



https://slurm.schedmd.com/quickstart.html

squeue

The **squeue** command is used to query and view information about the user jobs submitted to the Slurm scheduling queue.



In general, you should only need to check the status of the jobs you've submitted to Slurm.

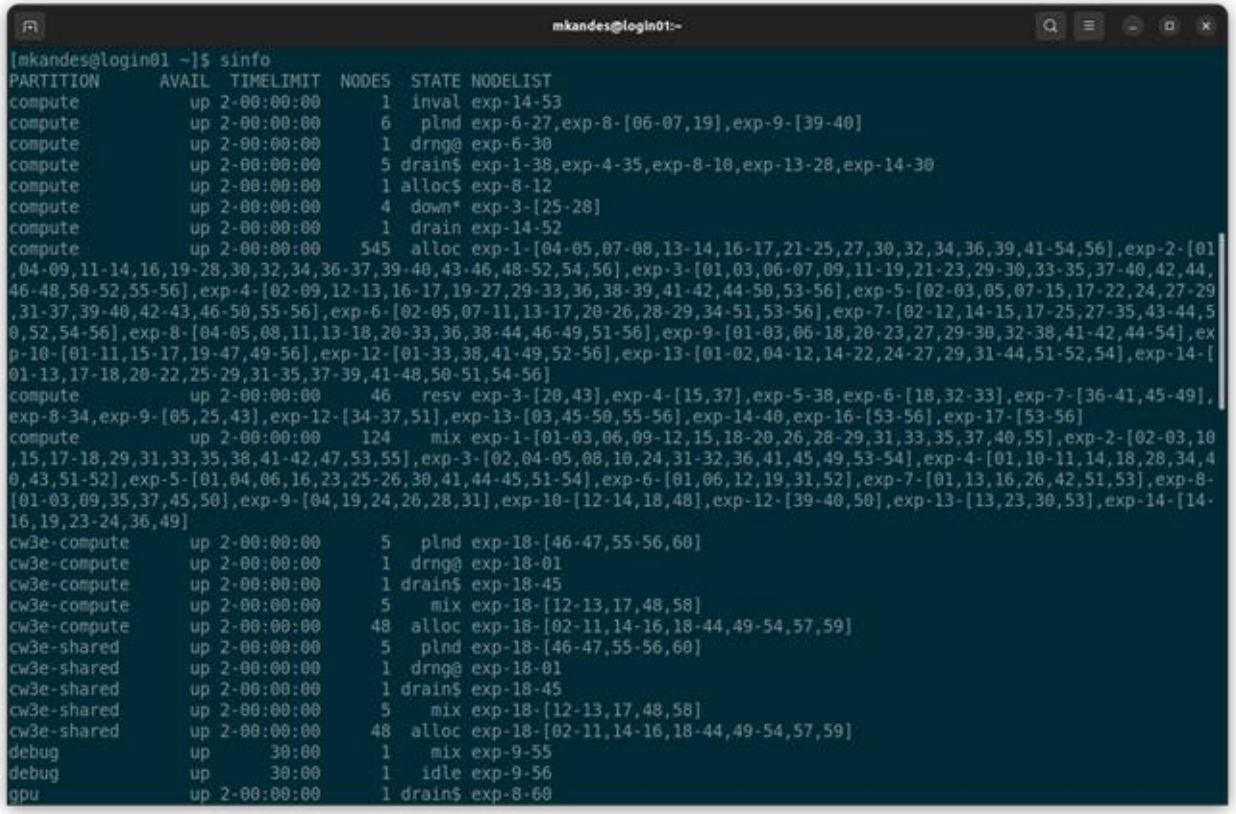
Recommendation: Create an alias in your shell's configuration file to only check the status of your jobs.

```
alias squeue="squeue -u ${USER}"
```

Do ***NOT*** run the `squeue` command incessantly to poll job status, either on purpose or on accident.

sinfo

The **`sinfo`** command is used to query and view information about the compute nodes and partitions managed by Slurm.



Slurm partitions are used to group nodes into logical, sometimes overlapping sets of nodes.

Each partition may have a unique set of constraints and features such as job size and runtime limits, types of resources available, user-based access permissions, etc.

You should always select the most appropriate partition for any job.

https://www.sdsc.edu/support/user_guides/expanse.html#running

Table of Contents: Part 1

- **High-Performance Computing Hardware and System Architecture**
- **Batch Job Scheduling and Resource Management**
- **Getting Started with the Slurm Workload Manager**
  - **`squeue`** - view information about ***jobs*** located in the Slurm scheduling ***queue***
  - **`sinfo`** - view information about Slurm ***nodes*** and ***partitions***
- **First Batch Job(s) with Slurm**
  - **`sbatch`** - submit a ***batch script*** to Slurm
  - **`scancel`** - used to signal or cancel jobs, job arrays or job steps
  - **`sacct`** - displays accounting data for all jobs in the Slurm database
- **Best Practices with Slurm**

Batch Scripts Contents

```
[uswername@login02 calc-prime]$ cat mpi-prime-slurm.sb
#!/bin/bash
#SBATCH --job-name="mpi_prime"
#SBATCH --output="mpi_prime.%j.%N.out"
####SBATCH --partition=compute
#SBATCH --partition=debug
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --export=ALL
#SBATCH -t 00:10:00
#SBATCH -A use300

## Environment
module purge
module load slurm
module load cpu
module load gcc/10.2.0
module load openmpi/4.0.4

## echo job name and id:
echo "SLURM_JOB_NAME:  $SLURM_JOB_NAME"
echo "SLURM_JOB_ID:    $SLURM_JOB_ID"
d=`date`
echo "DATE:  $d"
```

Batch Script 4 Main Parts:

- "Shebang" !
  - #!/bin/bash command tells the shell to run the script using the bash
- Resource Request:
  - options preceded with "#SBATCH" before any executable commands in the script
- Dependencies
  - Loads software that the project depends on to execute
- Job Steps:
  - Specify the list of tasks to be carried out.

Batch Script Output: ENV_Info

```
[uswername@login02 env_info ]$ cat env-slurm.sb
#!/bin/bash
#SBATCH --job-name=" env_info "
#SBATCH --output="mpi_prime.%j.%N.out"
####SBATCH --partition=compute
#SBATCH --partition=debug
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --export=ALL
#SBATCH -t 00:10:00
#SBATCH -A use300

## Environment
module purge
module load slurm
module load cpu
module load gcc/10.2.0
module load openmpi/4.0.4

## echo job name and id:
echo "SLURM_JOB_NAME:  $SLURM_JOB_NAME"
echo "SLURM_JOB_ID:    $SLURM_JOB_ID"
d=`date`
echo "DATE:  $d"
e=`env`
echo "env= $e"
```

```
[mthomas@login02 env_info ]$ sbatch env-slurm.sb
Submitted batch job 14126259
[mthomas@login02 env_info]$ !sq
squeue -u mthomas
JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
14126259     debug env_info   mthomas  R       0:04      1 exp-9-55
[mthomas@login01 env_info ]$ cat env_info.14126259.exp-4-35.out
SLURM_JOB_NAME:  env_info
SLURM_JOB_ID:    14126259
hostname=  exp-4-35
date=  Sun Jun 26 22:05:15 PDT 2022
whoami=  mthomas
pwd=  /home/mthomas/hpctr-examples/env_info
Currently Loaded Modules:       1) slurm/expanse/21.08.8   2) cpu/0.15.4
------------------------------
env= LD_LIBRARY_PATH=/cm/shared/apps/slurm/current/lib64/slurm:
[SNIP]
/cm/shared/apps/slurm/current/lib64
SLURM_SUBMIT_DIR=/home/mthomas/hpctr-examples/env_info
HISTCONTROL=ignoredups
DISPLAY=localhost:16.0
HOSTNAME=exp-4-35
[SNIP]
```

General Steps: Compiling/Running Jobs

- Change to your working directory (e.g. hpctr-examples):

```
[mthomas@login02 ~]$
[mthomas@login02 ~]$ cd /home/$USER/hpctr-examples/mpi
[mthomas@login02 mpi]$ ll hello_mpi.f90
-rw-r--r-- 1 mthomas use300 333 Feb 18 13:42 hello_mpi.f90
[mthomas@login02 mpi]$
```

```
[mthomas@login02 ~]$ module list
Currently Loaded Modules:
  1) shared   2) cpu/0.15.4   3) slurm/expanse/21.08.8
  4) sdsc/1.0   5) DefaultModules
```

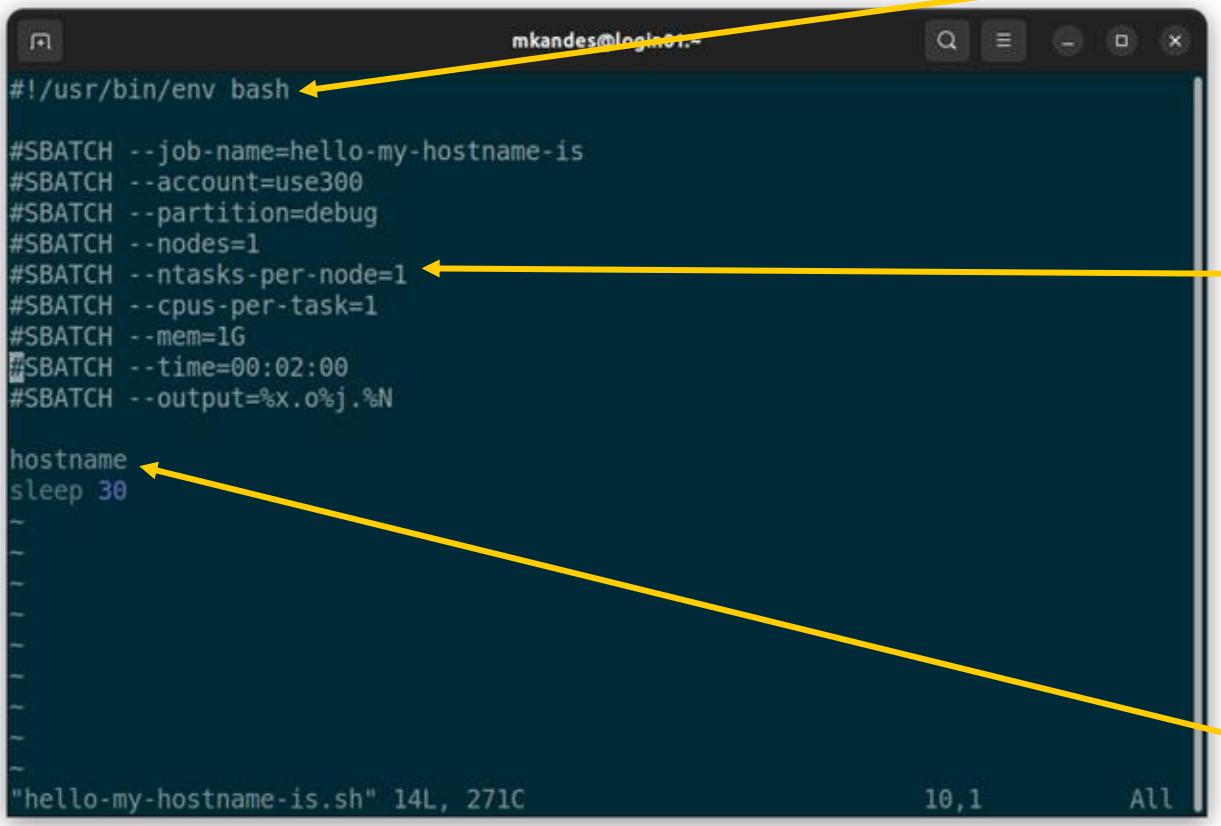
- Compile MPI hello world code:

```
[mthomas@login02 mpi]$
[mthomas@login02 mpi]$ mpif90 -o hello_mpi hello_mpi.f90
[mthomas@login02 mpi]$
```

```
[mthomas@login02 mpi]$
[mthomas@login02 mpi]$ ls -lt hello_mpi
-rwxr-xr-x 1 mthomas use300 22440 Jun 26 18:45 hello_mpi
[mthomas@login02 mpi]$
```

```
[mthomas@login02 mpi]$
[mthomas@login02 mpi]$ sbatch hellompi-slurm.sb
Submitted batch job 14124495
[mthomas@login02 mpi]$
```

Exercise 1: Hello, my hostname is



Exercise materials are available at https://github.com/sdsc-complecs/batch-computing

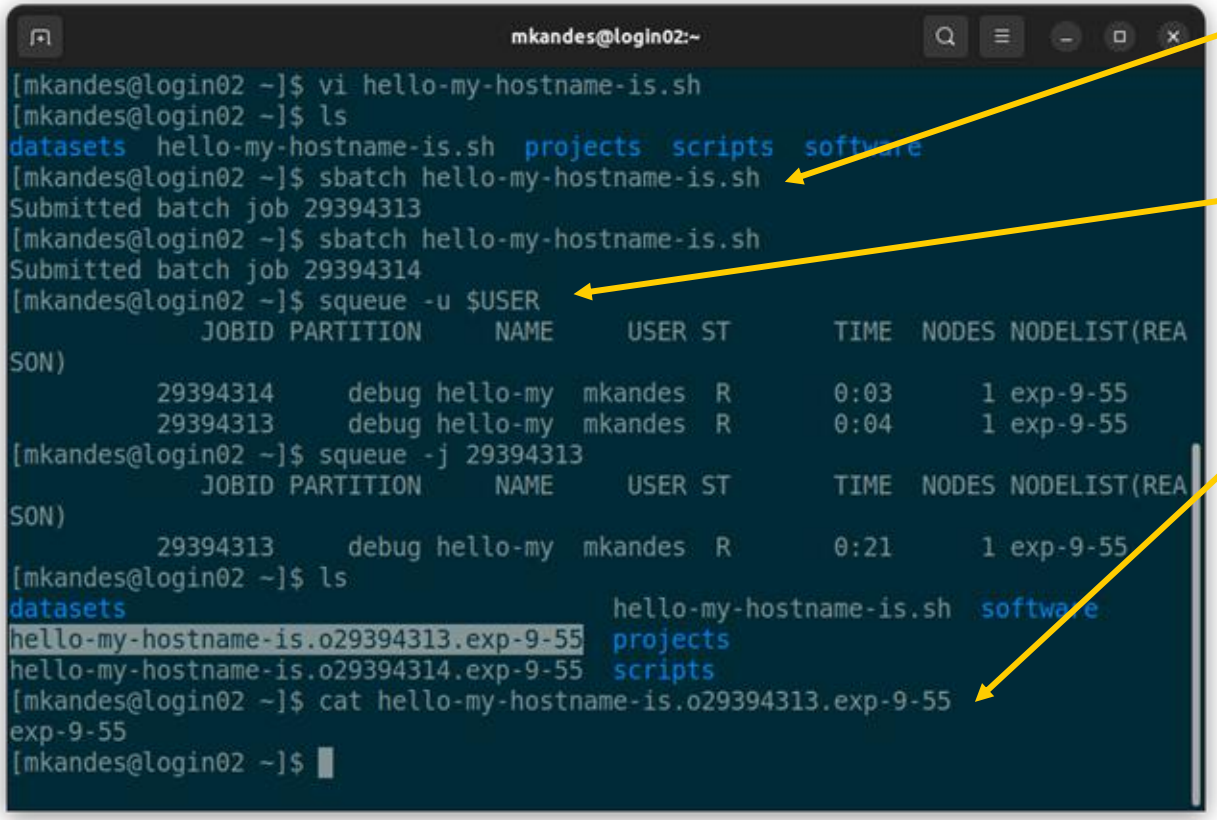
shebang (#!) character sequence is used to specify an interpreter that will execute the commands present in a plain text file.

#SBATCH directives are used to specify the computational resource requirements for your job and various other attributes of it. e.g., name of the job and output files, which account to charge the job to, etc.

All executable commands to be run must follow the list of #SBATCH directives specified in any job script.

sbatch

The **sbatch** command is used to submit a batch job script to Slurm.



Upon submission, every batch job is assigned a `SLURM_JOB_ID` number.

After submission, you can monitor the status of your jobs with the squeue command.

While a job runs, it will write standard output (and error) data to the file(s) named in your #SBATCH directives.

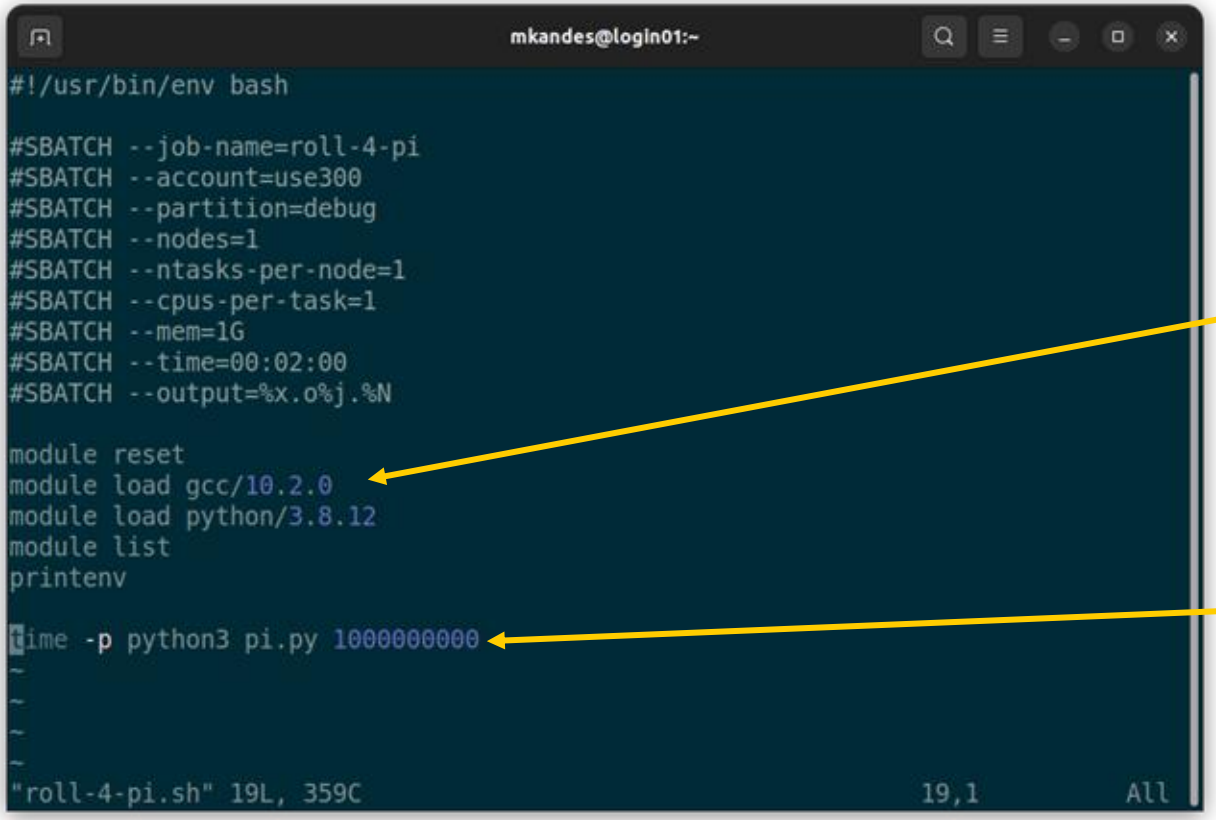
```
#SBATCH --output=%x.o%j.%N
```

`%x = SLURM_JOB_NAME`

`%j = SLURM_JOB_ID`

`%N = hostname -s`

Exercise 2: Roll 4 Pi



Exercise materials are available at https://github.com/sdsc-complecs/batch-computing

In most cases, you will want to follow the #SBATCH directives in your job script with a set of commands that configure the software environment in which the primary executable will run.

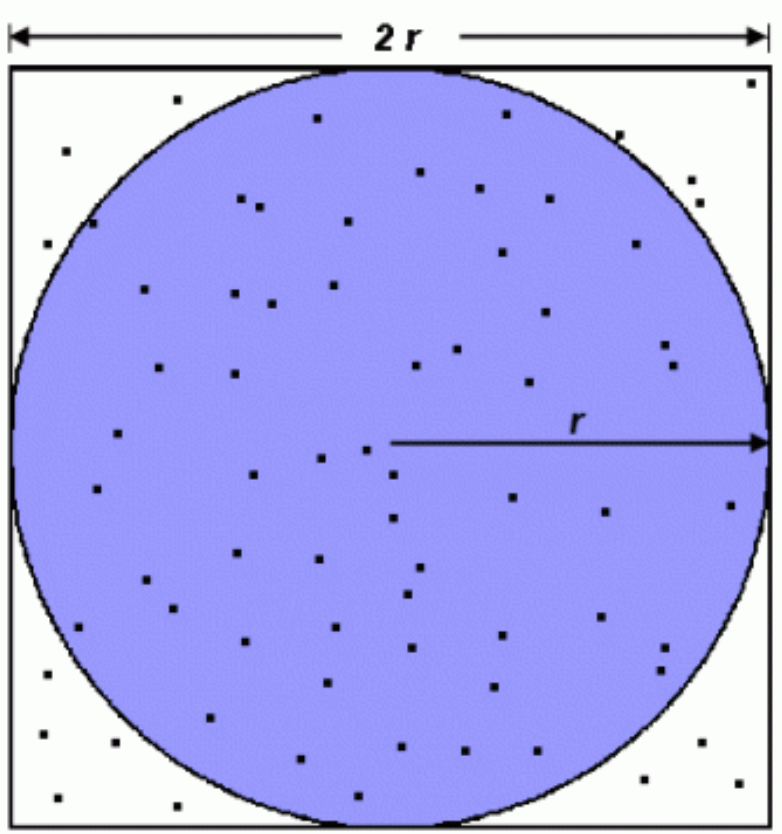
On many HPC systems, you will configure the software environment using pre-compiled and installed software modules.

Here, we estimate the value of π using a Monte Carlo method implemented in Python.

The prepended **`time`** command measures the runtime of the estimation.

sbatch and roll (100M samples)

```
mkandes@login02:~
[mkandes@login02 ~]$ ls
datasets                                hello-my-hostname-is.sh  roll-4-pi.sh
hello-my-hostname-is.o29394313.exp-9-55  pi.py                    scripts
hello-my-hostname-is.o29394314.exp-9-55  projects                 software
[mkandes@login02 ~]$ sinfo -p debug
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
debug        up      30:00      1   plnd exp-9-56
debug        up      30:00      1    mix exp-9-55
[mkandes@login02 ~]$ sbatch roll-4-pi.sh
Submitted batch job 29402750
[mkandes@login02 ~]$ squeue -u $USER
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REA
SON)
          29402750     debug roll-4-p  mkandes  R       0:22      1 exp-9-55
[mkandes@login02 ~]$ squeue -u $USER
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REA
SON)
[mkandes@login02 ~]$ tail -n 4 roll-4-pi.o29402750.exp-9-55
3.1413662714136628
real 62.54
user 62.36
sys 0.01
[mkandes@login02 ~]$
```



$$A_S = (2r)^2 = 4r^2$$

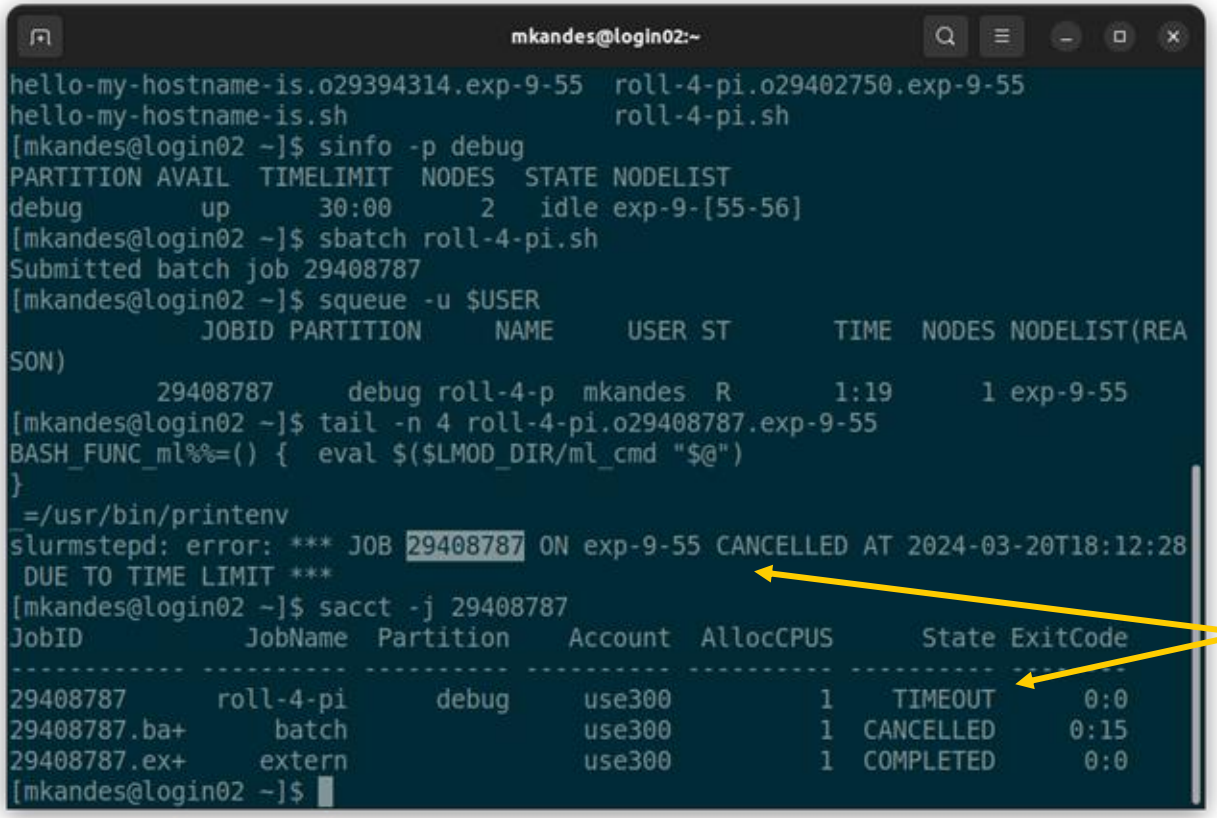
$$A_C = \pi r^2$$

$$\pi = 4 \times \frac{A_C}{A_S}$$

What do you expect will happen if we increase the number of samples to one billion and resubmit the job??

sacct

The `sacct` command displays job data from the Slurm accounting database.



What can you do when you submit a job you later realize has a mistake and/or might fail?

Use to look up information about completed and/or failed jobs when they are no longer visible in `squeue`.

Quickly check the final state of a job and answer these questions: Did the job run and exit normally? If not, does Slurm know why it failed?

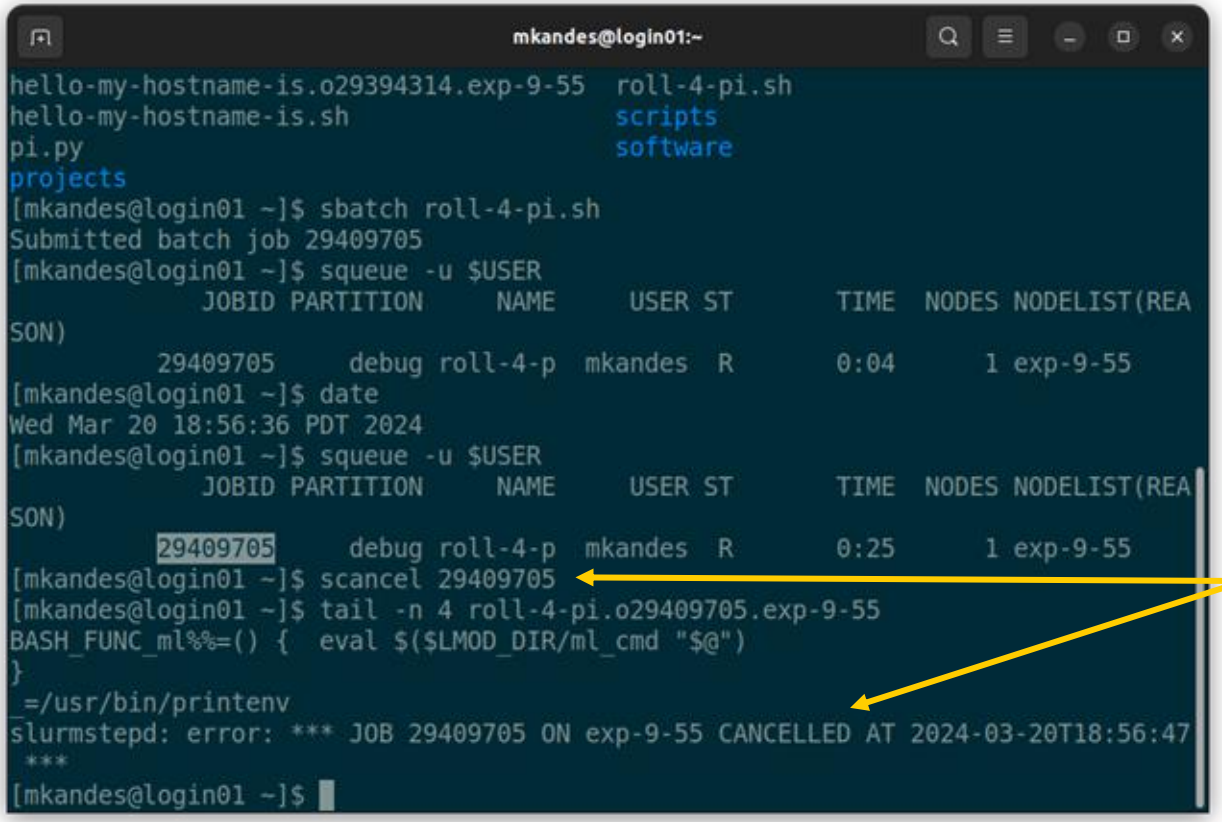
One of the most common failure modes for any batch job on an HPC system is running out of time.

Do ***NOT*** run extensive `sacct` queries without good reason.

slurm workload manager

scancel

The **scancel** command is used to signal or cancel jobs, job arrays or job steps.



Use whenever you want to remove a pending job from the squeue or stop a running job.

Table of Contents: Part 1

Parallel Computing Myth #2: Scheduler Variant

Simply requesting more compute resources from your batch job scheduler will automatically reduce the time to solution for your problem.

WRONG!

The truth is your application must be designed to take advantage of the additional compute resources you've requested. If it is designed to do so, then you generally must still configure your batch job to correctly utilize those resources.

Prerequisite Knowledge:

- Parallel Computing Concepts

Exercise 3: Build Job, Run Job

If you need to compile and install software on an HPC system, then you should consider writing a batch job to build and install the software instead of proceeding to do so interactively from the command-line.

There are a number of advantages of using this approach:

- build job script will document how to build and install the software
- build and install logs will be available in the job's standard output (and/or error) files
- software may need to be compiled on a specific compute node architecture
- software environment created within the build job to compile the software will need to be replicated in the job scripts that eventually run the software anyway

Exercise materials are available at https://github.com/sdsc-complecs/batch-computing

Parallel Pi with OpenMP

Parallel computing jobs can take advantage of more CPU resources

mkandes@login01:~

```
#!/usr/bin/env bash

#SBATCH --job-name=build-pi-omp
#SBATCH --account=use300
#SBATCH --partition=debug
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G
#SBATCH --time=00:02:00
#SBATCH --output=%x.o%j.%N

module reset
module load gcc/10.2.0
module list
printenv

make
~
~
~
~
~
"build-pi-omp.sh" 18L, 308C                                   18,1          All
```

mkandes@login01:~

```
#!/usr/bin/env bash

#SBATCH --job-name=run-pi-omp
#SBATCH --account=use300
#SBATCH --partition=debug
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=1G
#SBATCH --time=00:02:00
#SBATCH --output=%x.o%j.%N

module reset
module load gcc/10.2.0
module list
export OMP_NUM_THREADS="${SLURM_CPUS_PER_TASK}"
printenv

time -p ./pi_omp.x -s 10000000000
~
~
~
~
"run-pi-omp.sh" 19L, 383C                                     8,1           All
```

Build Job

Run Job

Use Slurm environment variables!

Exercise 4: GPU-Accelerated Workloads

GPUs are now a first-class compute resource in Slurm

Don't forget to define your own environment variables as needed!

Your job's software environment can be a mix of standard HPC software modules and alternative software package management methods. e.g., Singularity containers, conda environments

Exercise materials are available at https://github.com/sdsc-complecs/batch-computing

Exercise 5: Multinode MPI Jobs

You can request multiple compute nodes for your jobs. But only do so if the software you're running supports distributed memory parallelism, e.g., MPI

```
mkandes@login01:~
#!/usr/bin/env bash

#SBATCH --job-name=gromacs-2020.4-2ufeq67-aocc-3.2.0-io3s466-openmpi-4.1.3-xigazqd-water-cut1.0_GMX50_bare-3072-4-node-128-mpi-1-omp
#SBATCH --account=use300
#SBATCH --partition=compute
#SBATCH --nodes=4
#SBATCH --ntasks-per-node=128
#SBATCH --cpus-per-task=1
#SBATCH --mem=243G
#SBATCH --time=00:05:00
#SBATCH --output=%x.o%j.%N

declare -xr COMPILER_NAME='aocc'
declare -xr COMPILER_VERSION='3.2.0'
declare -xr COMPILER_VARIANT='io3s466'
declare -xr COMPILER_MODULE="${COMPILER_NAME}/${
declare -xr MPI_NAME='openmpi'
declare -xr MPI_VERSION='4.1.3'
declare -xr MPI_VARIANT='xigazqd'
declare -xr MPI_MODULE="${MPI_NAME}/${MPI_VERSION
declare -xr APPLICATION_NAME='gromacs'
```

```
mkandes@login01:~
  if [[ ! -f "${SLURM_SUBMIT_DIR}/benchmarks/${GROMACS_BENCHMARK_TARBALL}" ]]; then
    wget "${GROMACS_ROOT_URL}/benchmarks/${GROMACS_BENCHMARK_TARBALL}"
  fi
  tar -xzvf "${GROMACS_BENCHMARK_TARBALL}"
fi

cd "${SLURM_SUBMIT_DIR}"

time -p mpirun -n 1 gmx_mpi grompp \
  -f "${GROMACS_BENCHMARK_DATA_DIR}/pme.mdp" \
  -c "${GROMACS_BENCHMARK_DATA_DIR}/conf.gro" \
  -p "${GROMACS_BENCHMARK_DATA_DIR}/topol.top" \
  -po "mdout.${SLURM_JOB_NAME}.${SLURM_JOB_ID}.mdp" \
  -o "topol.${SLURM_JOB_NAME}.${SLURM_JOB_ID}.tpr"

time -p mpirun -n "${SLURM_NTASKS}" --bind-to core --map-by ppr:1:core:PE=1 \
  --display-allocation --display-map --report-bindings --verbose gmx_mpi mdrun \
  -nb cpu -pme cpu -bonded cpu -pin on -resethway -noconfout -nsteps 10000 \
  -s "topol.${SLURM_JOB_NAME}.${SLURM_JOB_ID}.tpr" \
  -cpo "state.${SLURM_JOB_NAME}.${SLURM_JOB_ID}.cpt" \
  -e "ener.${SLURM_JOB_NAME}.${SLURM_JOB_ID}.edr" \
  -g "md.${SLURM_JOB_NAME}.${SLURM_JOB_ID}.log" \
  -v
                                                                77,3          Bot
```

Exercise materials are available at https://github.com/sdsc-complecs/batch-computing

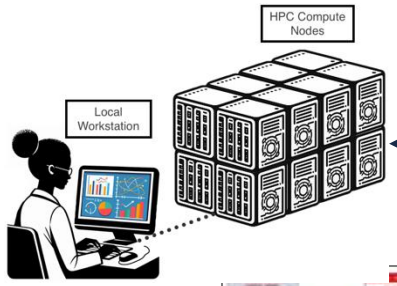
But again, you will still likely need to configure the call to the application to use the requested resources correctly.

Table of Contents: Part 2

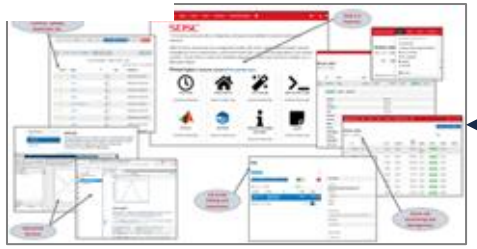
HPC Jobs can be run as batch jobs (background) or interactively (real time)

Terminal shell – submit batch

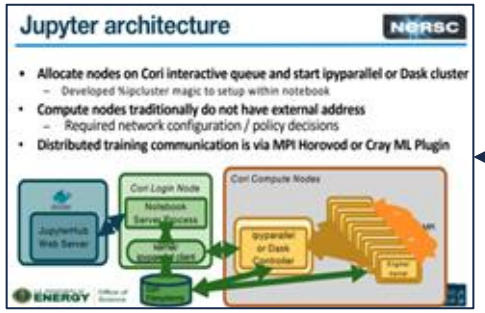
Img src: https://hpc.arizona.edu/

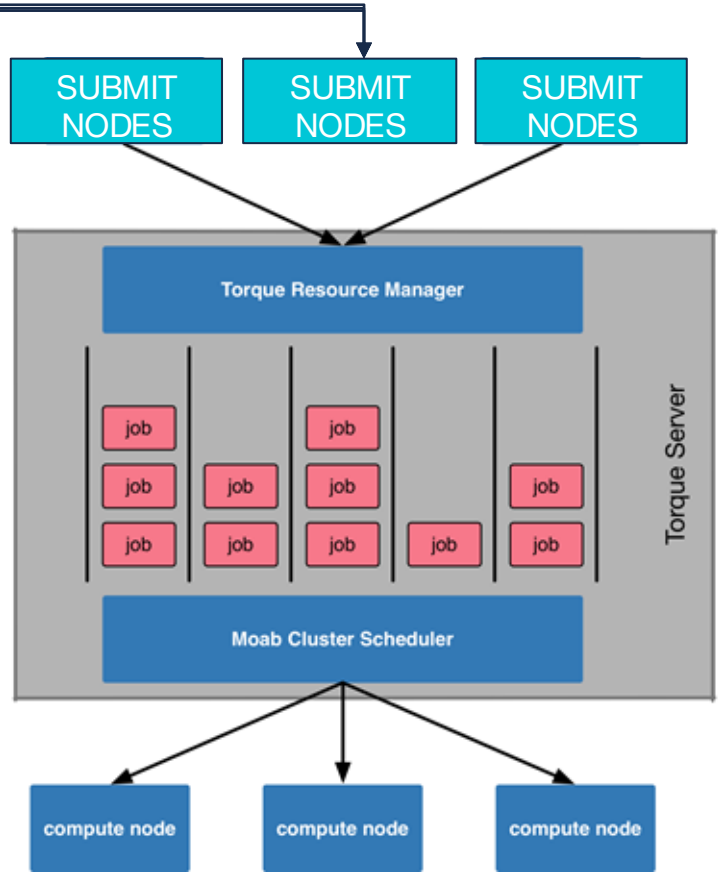


Interactive: https://portal.expanse.edu



Interactive Computing with Jupyter





Src: https://hpc.dccn.nl/_images/torque_moab_arch.png

What is Interactive HPC Computing

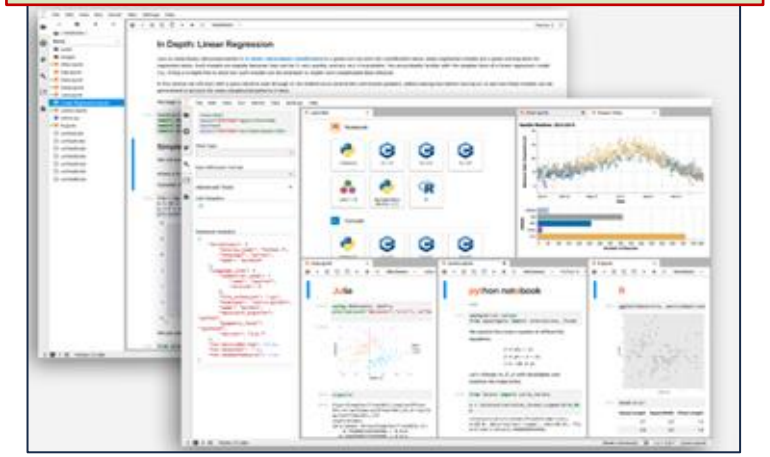
- In **computer** science, **interactive computing** refers to software which accepts input from the user as it runs.
  - **Interactive** software includes commonly used programs, such as word processors or spreadsheet applications.
- **Interactive HPC computing** involves *real-time* user inputs to perform tasks on a set of compute node(s) including:
  - Code development, real-time data exploration, and visualizations.
  - Used when applications have large data sets or are too large to download to local device, software is difficult install, etc.
  - User inputs come via command line interface or application GUI (Jupyter Notebooks, Matlab, R-studio).
  - Actions performed on remote compute nodes as a result of user input or program out.

Interactive HPC Computing- Motivation

- Need more memory: Your jobs no longer fit onto the CPU/system you have been using:
  - That is why Expanse has 768 nodes (128 cores per node), with 256 GB DDR memory on each node
  - My MacBook Pro: 1 node, 8 cores, 16 GB DDR
- Too much data: your application needs more room:
  - Expanse has 1TB NVME/node, and 12 PB file system.
  - My MBP: 500 GB, no NVME
- Your network is too slow:
  - Expanse has connections to ~ 150GB/sec (or faster) networks
  - My MBP: 300 Mbps download; 11 Mbps upload
  - Bioinfo lab, running analysis on PC: FastQ dataset ~ 500 MB: would need 360+ seconds to upload 1 run.

Interactive HPC Computing

Jupyter Notebook: https://jupyter.org/



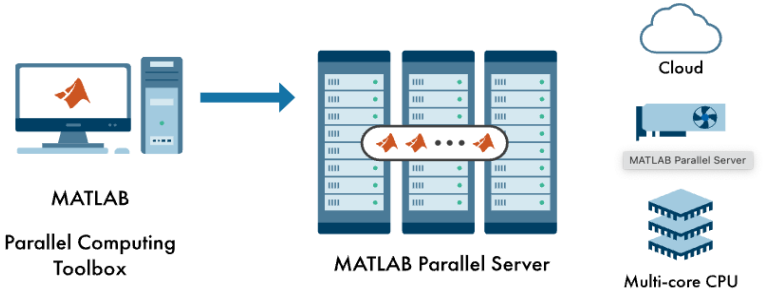
Expanse User Portal

https://portal.expanse.sdsc.edu

Parallel Matlab (Mathworks)

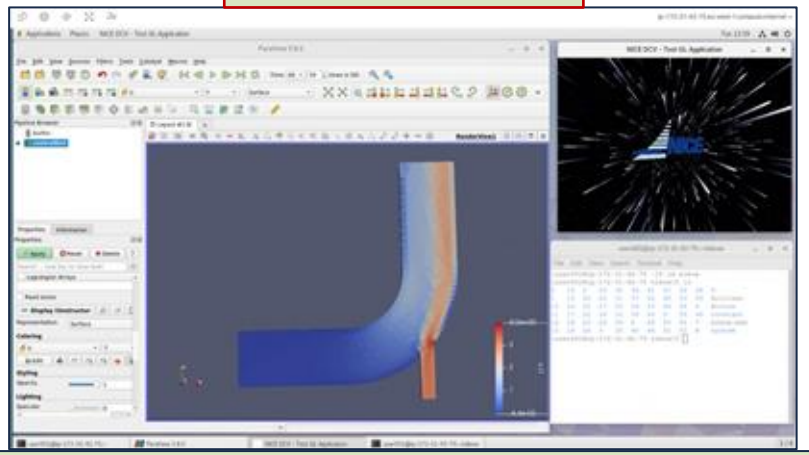
```
parpool("HPC",1000);

parfor ii = 1:3000
    c(ii,:) = eig(rand(1000));
end
```



https://azuremarketplace.microsoft.com/en-us/marketplace/apps/mathworks-inc.matlab-parallel-server-listing?tab=Overview

Paraview (AWS)



https://aws.amazon.com/blogs/compute/how-to-run-3d-interactive-applications-with-nice-dcv-in-aws-batch/

Examples of Interactive HPC Methods & Applications

- Applications/Data Analysis Platforms:
  - Support interactions from within the code/model using libraries
  - Matlab, R, Jupyter Notebooks
  - NetCDF, HPF, TAU, ParaView
  - other
- Interact with large data sets after job is done:
  - Unix: query file info, location, output, grep, awk, sed
  - Cat the file contents from batch job or raw data
  - NetCDF data browser

Note: Interactive Services have a Key Vulnerability: They Provide Access to HPC File Systems, often over HTTP

SDSC Interactive Services Policy:

- Portals, JupyterHub, and other services cannot be mounted directly to disk (must be on VM or external):
  - Many use root in vulnerable ways
  - If a user launches Jupyter Lab or Notebooks, the jobs will be killed.
- Applications cannot run on login nodes.

SDSC recommendation:

- Use secure connections: when you choose insecure connections your account is vulnerable to hacking.
- Use portal.expanse.sdsc.edu

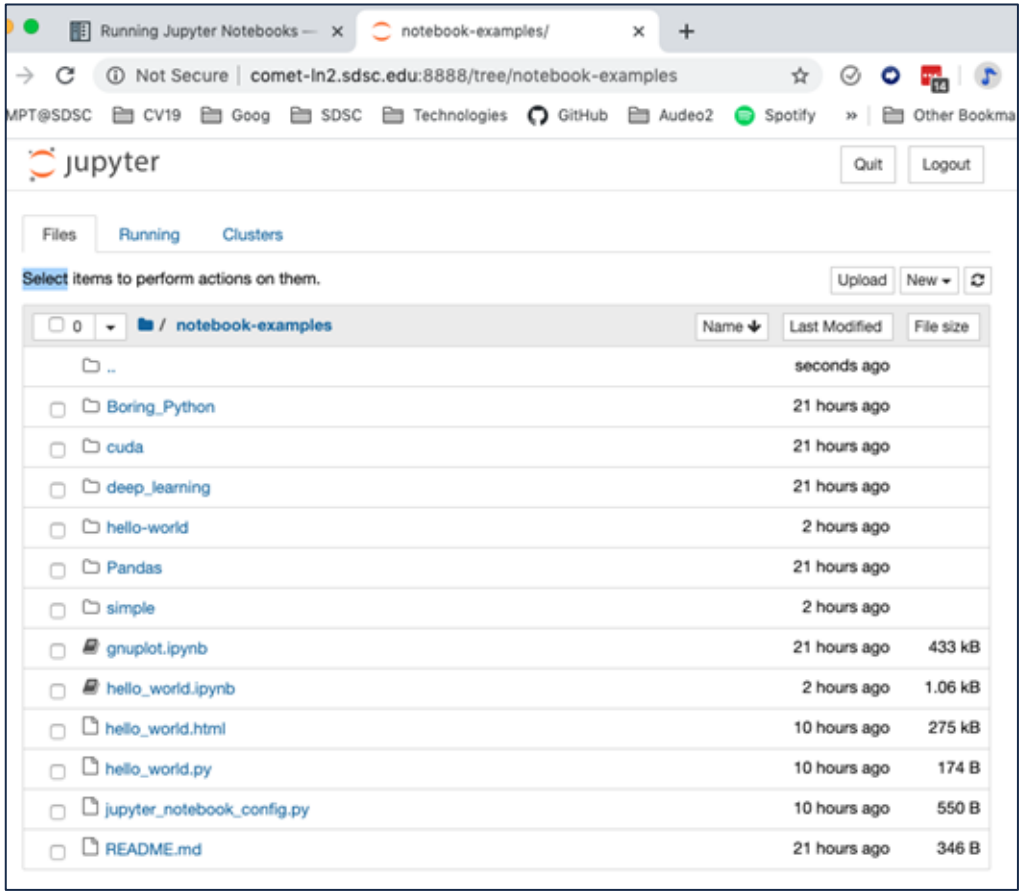


Table of Contents: Part 2

When would you need to use an Interactive node?

- The application is interactive (Matlab, Notebooks)
- Need to stage large amounts of data
- AI applications (Pytorch, Tensorflow)
- Need to compile a very large, complex application that may take a long time.
  - Java code, 1 million lines, > 30 minutes on DELL 5511 laptop

Accessing Interactive Compute Nodes on Expanse

- Connect to HPC system (e.g. Expanse) via terminal using *SSH* secure connections
- Use the *srun* command to obtain nodes for 'live,' command line interactive access:

CPU	srun --partition=debug --pty --account=abc123 --nodes=1 --ntasks-per-node=4 --mem=8G -t 00:30:00 --wait=0 --export=ALL /bin/bash
GPU	srun --partition=gpu-debug --pty --account=use300 --ntasks-per-node=10 --nodes=1 --mem=96G --gpus=1 -t 00:30:00 --wait=0 --export=ALL /bin/bash

(Tested 04/17/2024)

Using An Interactive CPU node

```
[mthomas@login01 calc-prime]$ srun --partition=debug  --pty --account=<<project>> --nodes=1 --ntasks-per-node=4 --mem=8G -t 00:30:00 --wait=0 --export=ALL /bin/bash

srun: job 24457429 has been allocated resources
[mthomas@exp-9-55 calc-prime]$ module purge
[mthomas@exp-9-55 calc-prime]$ module load slurm
[mthomas@exp-9-55 calc-prime]$ module load cpu
[mthomas@exp-9-55 calc-prime]$ module load gcc/10.2.0
[mthomas@exp-9-55 calc-prime]$ module load openmpi/4.1.1
[mthomas@exp-9-55 calc-prime]$ mpirun -n 16 ./mpi_prime
06 August 2023 11:10:26 PM
PRIME_MPI     n_hi= 5000000      C/MPI version
An MPI example program to count the number of primes: # processes is 64
        N       Pi          Time
        1        0       0.013258
        2        1       0.001058
        4        2       0.000101
        8        4       0.000101
    [SNIP]
   131072    12251       0.110848
   262144    23000       0.410792
   524288    43390       1.527210
  1048576    82025       5.733612
  2097152   155611      21.725862
PRIME_MPI - Master process: Normal end of execution.
06 August 2023 11:12:26 PM
```

Request an interactive node for 30 minutes

- Exit interactive session when your work is done or you will be charged more CPU time.
- Beware of oversubscribing your job: don't ask for more cores than you have requested.
- Intel compiler allows this, but your performance will be degraded.

https://www.sdsc.edu/systems/expanse/user_guide.html

Using Interactive GPU nodes

```
[snip]
Last login: Fri Feb 18 12:58:32 2022 from 76.176.117.51
[username @login02 ~]$
[username @login02 ~]$ srun --partition=gpu-debug --pty --account=use300 --ntasks-per-node=10 --nodes=1 --
mem=96G --gpus=1 -t 00:30:00 --wait=0 --export=ALL /bin/bash
srun: job 9794018 queued and waiting for resources
srun: job 9794018 has been allocated resources
[mthomas@exp-14-57 ~]$
[mthomas@exp-14-57 ~]$ nvidia-smi
Fri Feb 18 13:04:19 2022
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 460.32.03    Driver Version: 460.32.03    CUDA Version: 11.2     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla V100-SXM2...  On   | 00000000:86:00.0 Off |                    0 |
| N/A   34C    P0    41W / 300W |      0MiB / 32510MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
[username @exp-14-57 ~]$ exit
```

Request an interactive node for 30 minutes

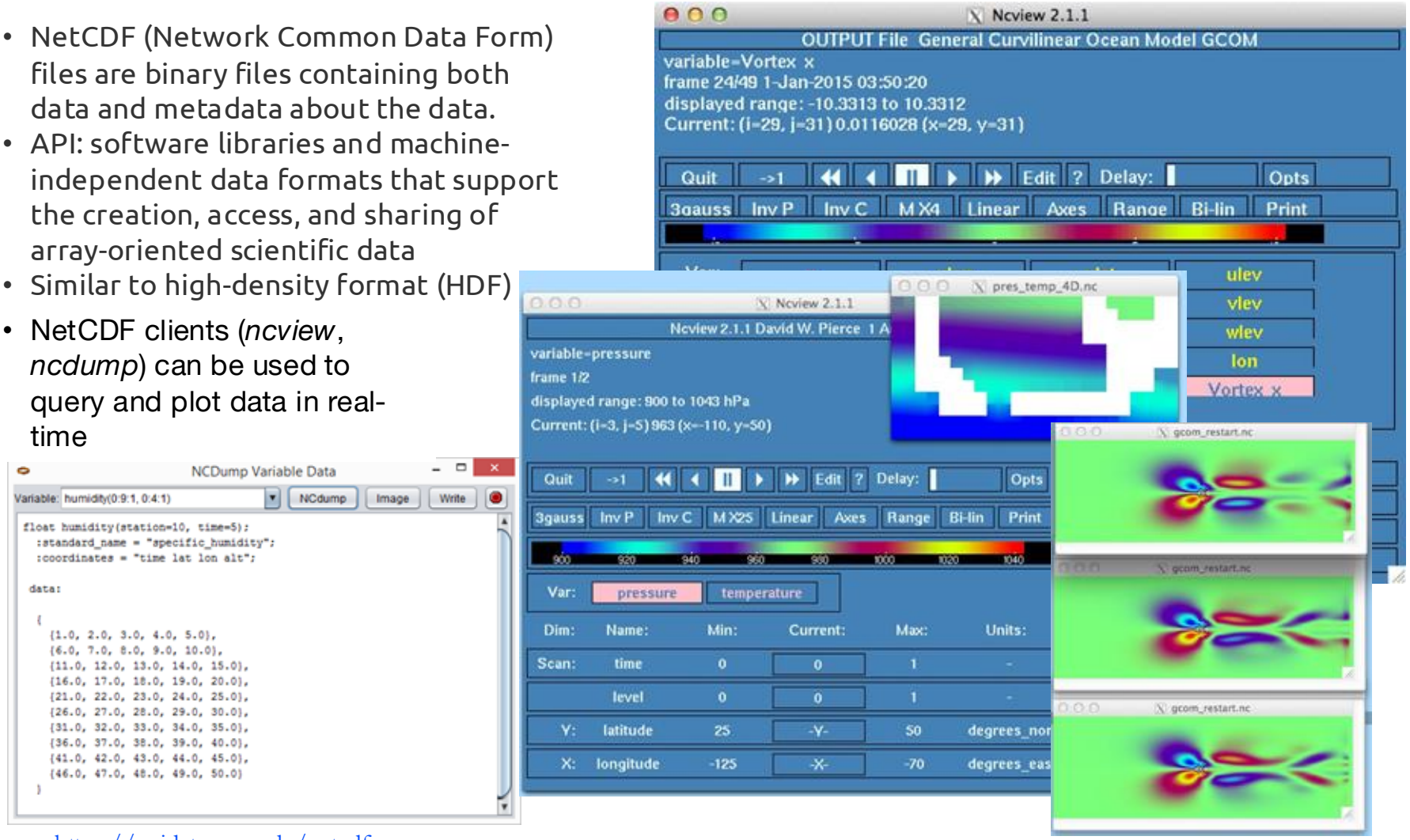
Verify you are on a GPU node

Exit when tasks are done

Table of Contents: Part 2

NetCDF Data Files

- NetCDF (Network Common Data Form) files are binary files containing both data and metadata about the data.
- API: software libraries and machine-independent data formats that support the creation, access, and sharing of array-oriented scientific data
- Similar to high-density format (HDF)
- NetCDF clients (*ncview*, *ncdump*) can be used to query and plot data in real-time



https://unidata.ucar.edu/netcdf

Outline

- What is High-Performance Computing (HPC)?
- What is HPC batch computing
- Defining Interactive HPC Computing
- Accessing Interactive HPC Nodes
- Interactive Application Examples
  - Viewing Data: unix file operators (grep, awk, cat), gnuplot, NetCDF
  - Programming & Visualization Platforms: Matlab, R, Jupyter Notebooks (and NetCDF)
  - Gateways & Portals: simplify access to interactive apps

Expanse Interactive Jobs: Running Matlab from Command Line

```
[mthomas@login01 ~]$  srun --partition=gpu-debug --pty --account=use300 --ntasks-per-node=10     --
nodes=1 --mem=96G --gpus=1 -t 00:30:00 --wait=0 --export=ALL /bin/bash
srun: job 14833549 queued and waiting for resources
srun: job 14833549 has been allocated resources
[mthomas@exp-9-55 ~]$ module load  gpu/0.17.3b
[mthomas@exp-9-55 ~]$ module load matlab/2022b/nmbr5dd
[mthomas@exp-9-55 ~]$ module list
Currently Loaded Modules:
  1) shared                3) sdsc/1.0        5) gpu/0.17.3b         (g)
  2) slurm/expanse/21.08.8 4) DefaultModules  6) matlab/2022b/nmbr5dd
```

Check that you have the right modules loaded

```
[mthomas@exp-9-55 ~]$ matlab
MATLAB is selecting SOFTWARE OPENGL
rendering.                                          < M A T L A B (R)>
  Copyright 1984-2022 The MathWorks, Inc.
     R2022a (9.12.0.1884302) 64-bit (glnxa64)
                February 16, 2022
 >> a = [1 3 5; 2 4 6; 7 8 10]
a =
    1    3    5
    2    4    6
    7    8   10
>> b=sin(a)
b =
   0.8415    0.1411   -0.9589
   0.9093   -0.7568   -0.2794
   0.6570    0.9894   -0.5440
>> plot(b)     -- nothing happens
>>
```

Expanse Interactive Jobs: Running Matlab with GUI

To use a Graphical User Interface (GUI) as part of your interactive job, you will need to set up Xforwarding. Example below is for using XQuartz on a MAC. For examples for Windows, see: http://systems.eecs.tufts.edu/x11-forwarding/

Step 1: *Log on to Expanse, using* ***–Y*** *option (trusted); create two connections; W2 should be the XQuartz terminal*

W1

```
[mthomas@home]$ ssh –Y mthomas@login.expanse.sdsc.edu
```

W2

```
[mthomas@home]$ ssh –Y mthomas@login.expanse.sdsc.edu
```

Step 2: Window 1: *Request an interactive node, using* ***srun command***
This window will not be used for anything else but holding the interactive node connection

W1

```
[mthomas@login01 ~]$ srun --partition=debug  --pty --account=use300 --
nodes=1 --ntasks-per-node=4  --mem=8G -t 00:30:00 --wait=0 --export=ALL
/bin/bash
srun: job 26814162 queued and waiting for resources
srun: job 26814162 has been allocated resources
[mthomas@exp-9-55 ~]$
```

Step 3: Window 2:
Connect to the interactive node: atlab

W2

```
[mthomas@login02 ~]$ ssh -Y exp-9-55
Last login: Wed Dec  6 19:23:47 2023 from 10.21.0.19
```

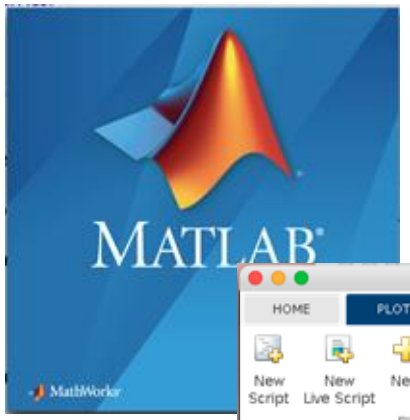
Step 4: Window 2:
Setup Matlab module environment
Launch Matlab

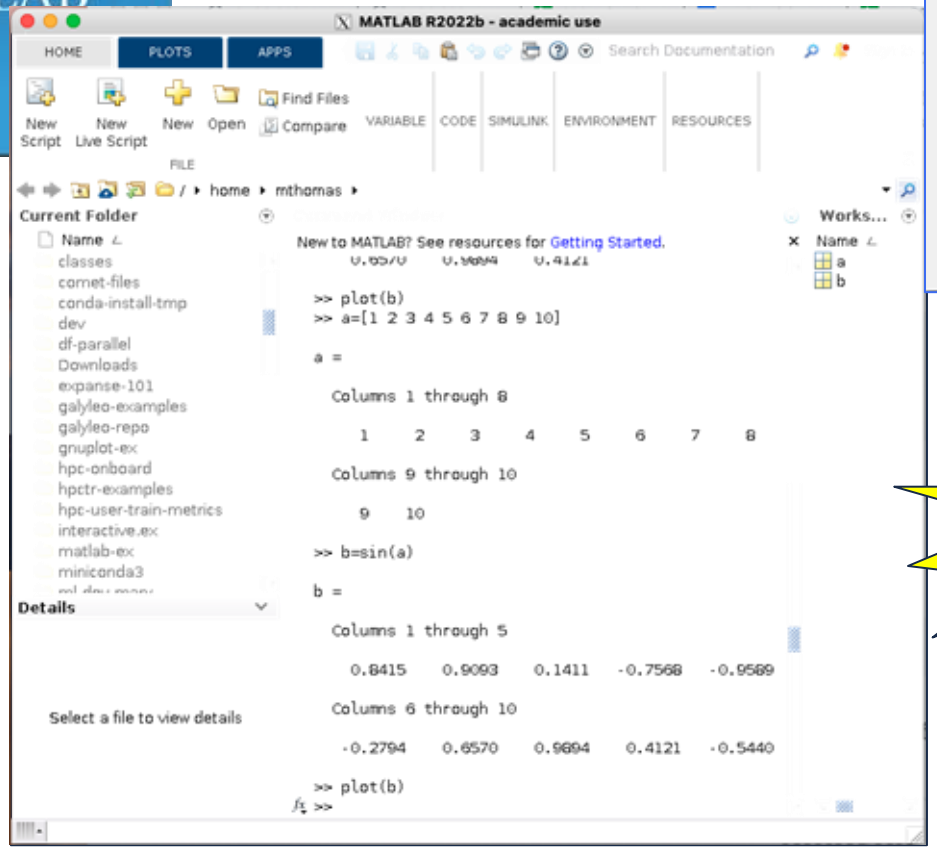
GUI : USE EXPANSE PORTAL

W2

```
[mthomas@exp-9-55 ~]$ module load  gpu/0.17.3b
[mthomas@exp-9-55 ~]$ module load matlab/2022b/nmbr5dd
[mthomas@exp-9-55 ~]$ module list
Currently Loaded Modules:
  1) shared     3) sdsc/1.0   5) gpu/0.17.3b  (g)
  2) slurm/expanse/21.08.8   4) DefaultModules   6) matlab/2022b/nmbr5dd
[mthomas@exp-9-55 ~]$ matlab
MATLAB is selecting SOFTWARE OPENGL rendering.
```

HPC Interactive Jobs: Running Matlab with GUI





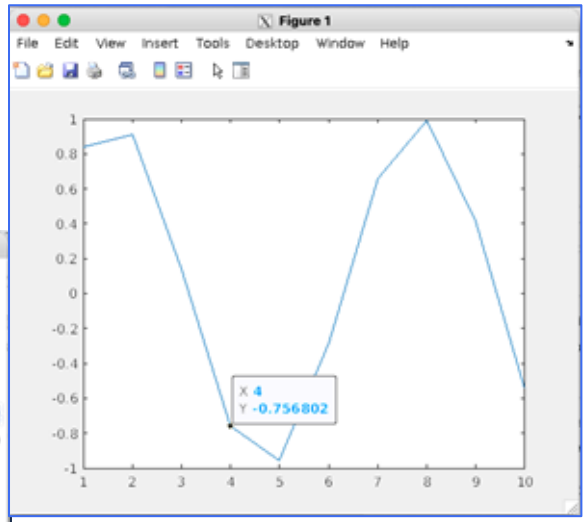


Table of Contents: Part 2

Juptyer Notebooks

What is Jupyter?

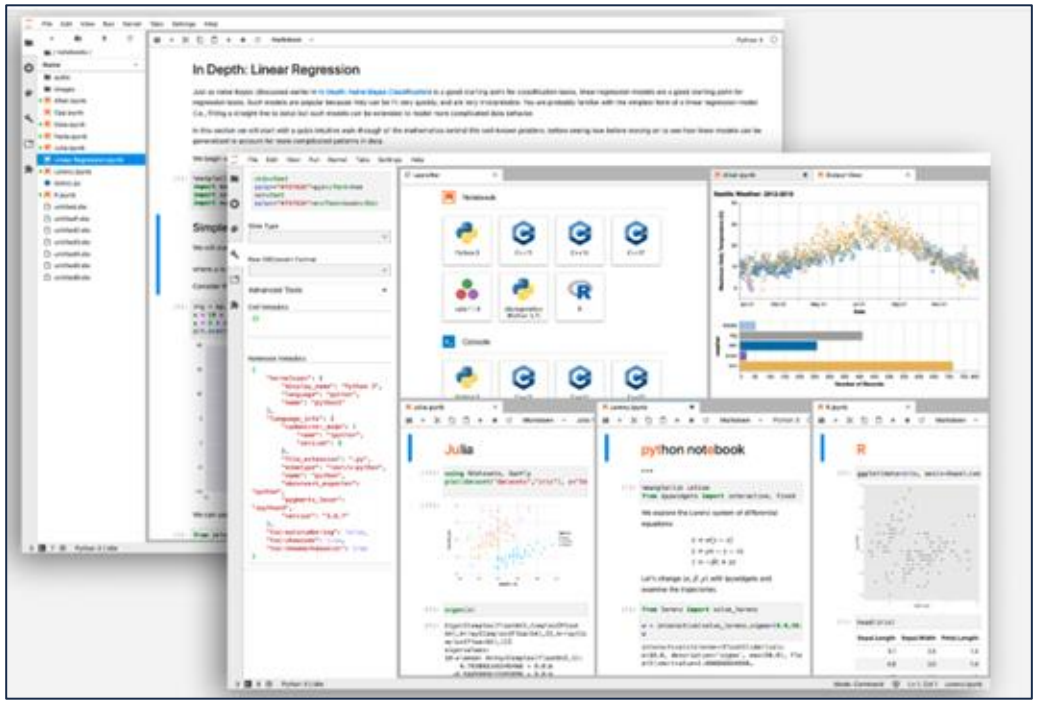
> ***Jupyter*** *is a free, open-source, interactive web tool known as a computational notebook, which researchers can use to combine software code, computational output, explanatory text and multimedia resources in a single document.* *(J. Perkel, https://www.nature.com/articles/d41586-018-07196-1)*

Common Jupyter Services:

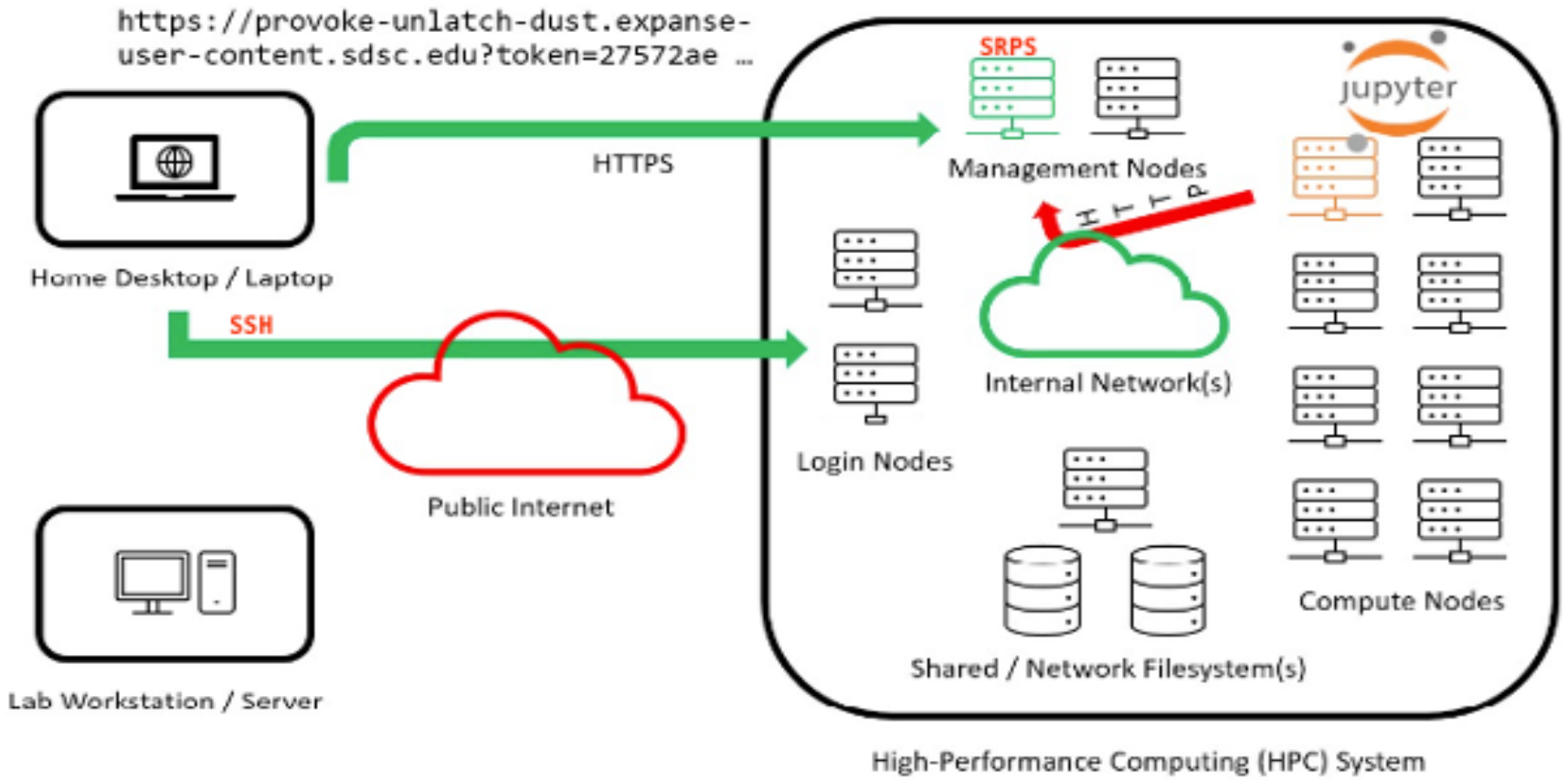
- Jupyter Notebooks (single user)
- JupyterLab: advanced version of notebook
- JupyterHub: multiuser Jupyter service

Running Secure Notebooks HPC Systems

- Install notebook application:
  - Locally: install Anaconda on your laptop (default is HTTP)
- Connecting local host (e.g. laptop) to HPC:
  - HTTP (default, insecure)
  - HTTP + SSH tunneling (secure, but inconvenient)
  - HTTPS + using the *Satellite Reverse Proxy Service* (SRPS) and *galyleo* client (secure, convenient)
- You can launch Jupyter services on SDSC:
  - CPU and GPUs
  - Interactive nodes: command line *or* Slurm batch script
- Treat the Notebook URL like a Password!



Running Notebooks Remotely & Securely Using SRPS & galyleo



Src: M Kandes: https://education.sdsc.edu/training/interactive/202112_running_jupyter_notebooks_on_expanse/

galyleo

- 2nd generation shell utility developed to orchestrate a user's interaction with both Satellite and Slurm to start a Jupyter session within a batch job.
- Developed while reviewing start-jupyter (prototype client) codebase to sort out how best to support Expanse (OOD) Portal and HPC User Services Group long-term; integrated into an existing SSH tunneling orchestration utility to use Satellite proxy service instead

- Key features in design:
  - HTTPS URL
  - Supports containers (Singularity on Expanse)
  - No need to install conda environment or update packages
  - Increases flexibility for users to configure software environment; but also try to makes it simpler for them to do this themselves
  - Batch job script is generated completely on-the-fly.
  - Command-line argument driven.
  - Quiet mode for OOD portal

https://github.com/mkandes/galyleo

Satellite Client: galyleo

Key features in design:

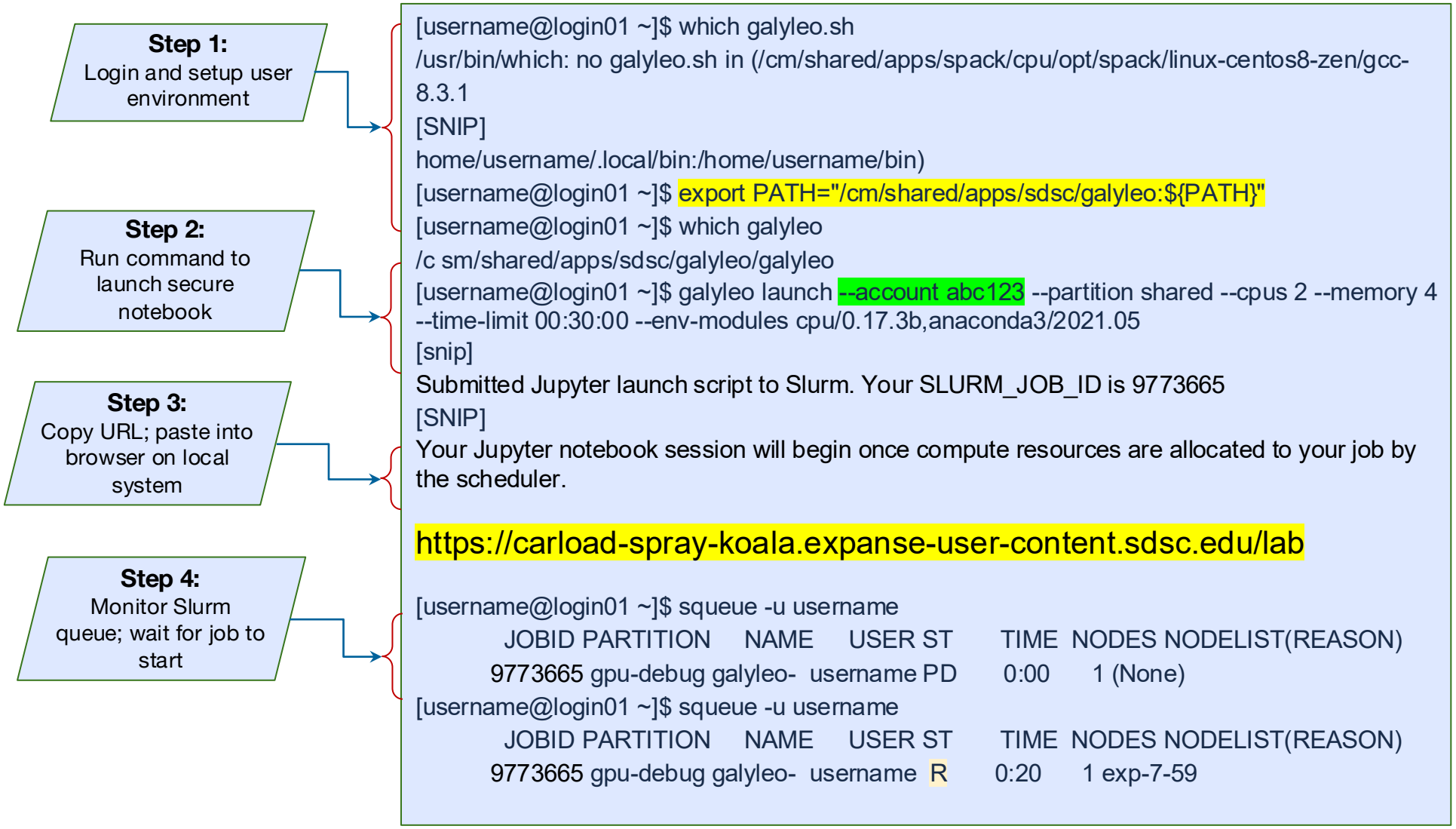
- User calls galyleo.sh launch script, which requests token from Satellite, passes token to batch job script and submits the job to Slurm; token redeemed from batch job once it runs
- Increase flexibility for users to configure software environment; but also try to make it simpler for them to do themselves
- Batch job script is generated completely on-the-fly.
- Command-line argument driven.
- Quiet mode for OOD portal

```
[username@login01 ~]export PATH="/cm/shared/apps/sdsc/galyleo:${PATH}"

[username@login01 ~]$ galyleo.sh --help
USAGE: galyleo.sh launch [command-line option] {value}
command-line option      : value
   -A | --account         :
   -R | --reservation     :
   -p | --partition       :
   -q | --qos             :
   -N | --nodes           :
   -n | --ntasks-per-node :
   -c | --cpus-per-task   :
   -M | --memory-per-node :  GB
   -m | --memory-per-cpu  :  GB
   -G | --gpus            :
      | --gres            :
   -t | --time-limit      :
   -j | --jupyter         :
   -d | --notebook-dir    :
   -r | --reverse-proxy   :
   -D | --dns-domain      :
   -s | --sif             :
   -B | --bind            :
      | --nv              :
   -e | --env-modules     :
      | --conda-env       :
   -Q | --quiet           : 1
```

https://https://github.com/sdsc/galyleo

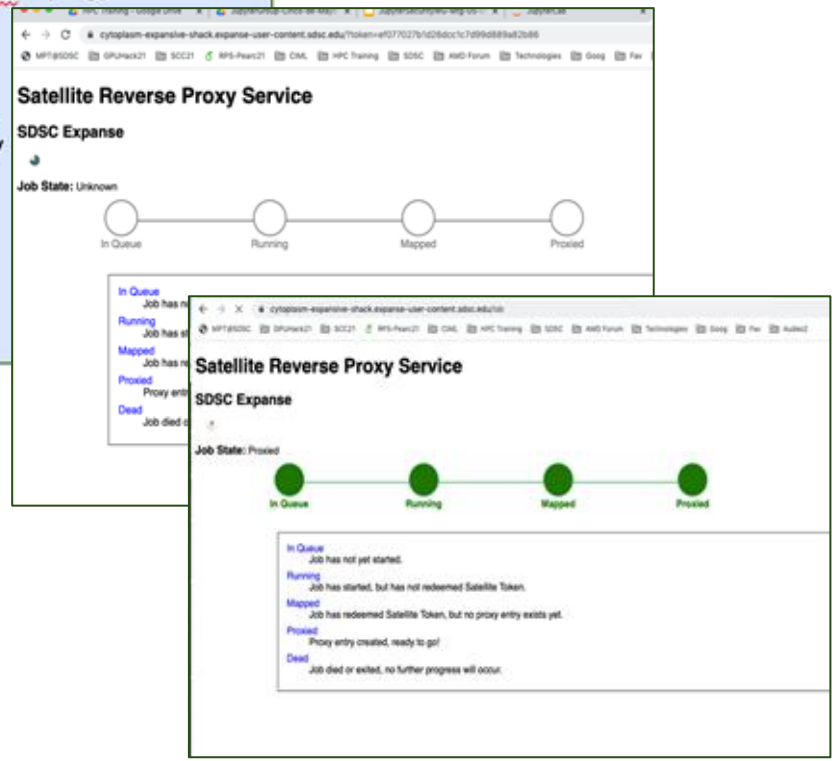
Launching CPU notebooks using galyleo

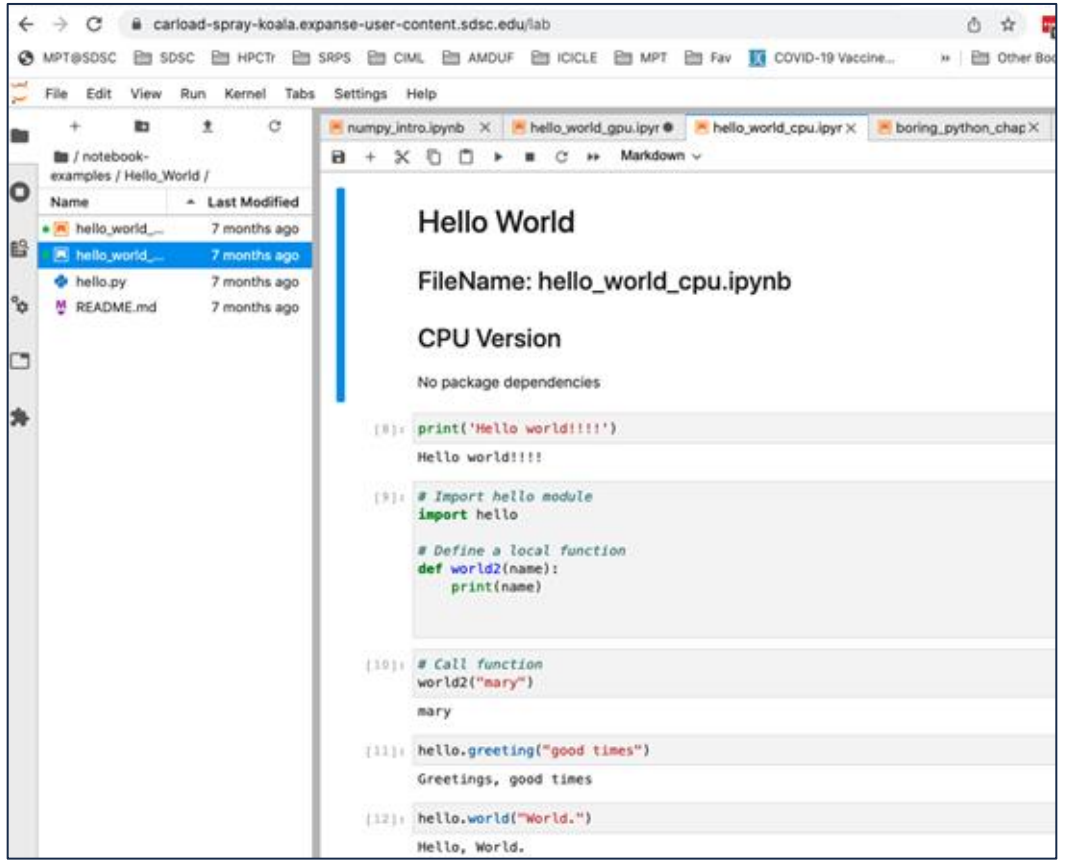


Satellite-Galyleo System

```
[username@login01 ~]export
PATH="/cm/shared/apps/sdsc/galyleo:${PATH}"

[username@login01 ~]$ galyleo.sh --help
USAGE: galyleo.sh launch [command-line
option] {value}
command-line option      : value
    -A | --account         :
    -R | --reservation     :
    -p | --partition       :
    -q | --qos             :
    -N | --nodes           :
    -n | --ntasks-per-node :
    -c | --cpus-per-task   :
    -M | --memory-per-node :  GB
    -m | --memory-per-cpu  :  GB
    -G | --gpus
       | --gres
    -t | --time-limit
    -j | --jupyter
    -d | --notebook-dir
    -r | --reverse-proxy
    -D | --dns-domain
    -s | --sif
    -B | --bind
       | --nv
    -e | --env-modules
       | --conda-env
    -Q | --quiet
```

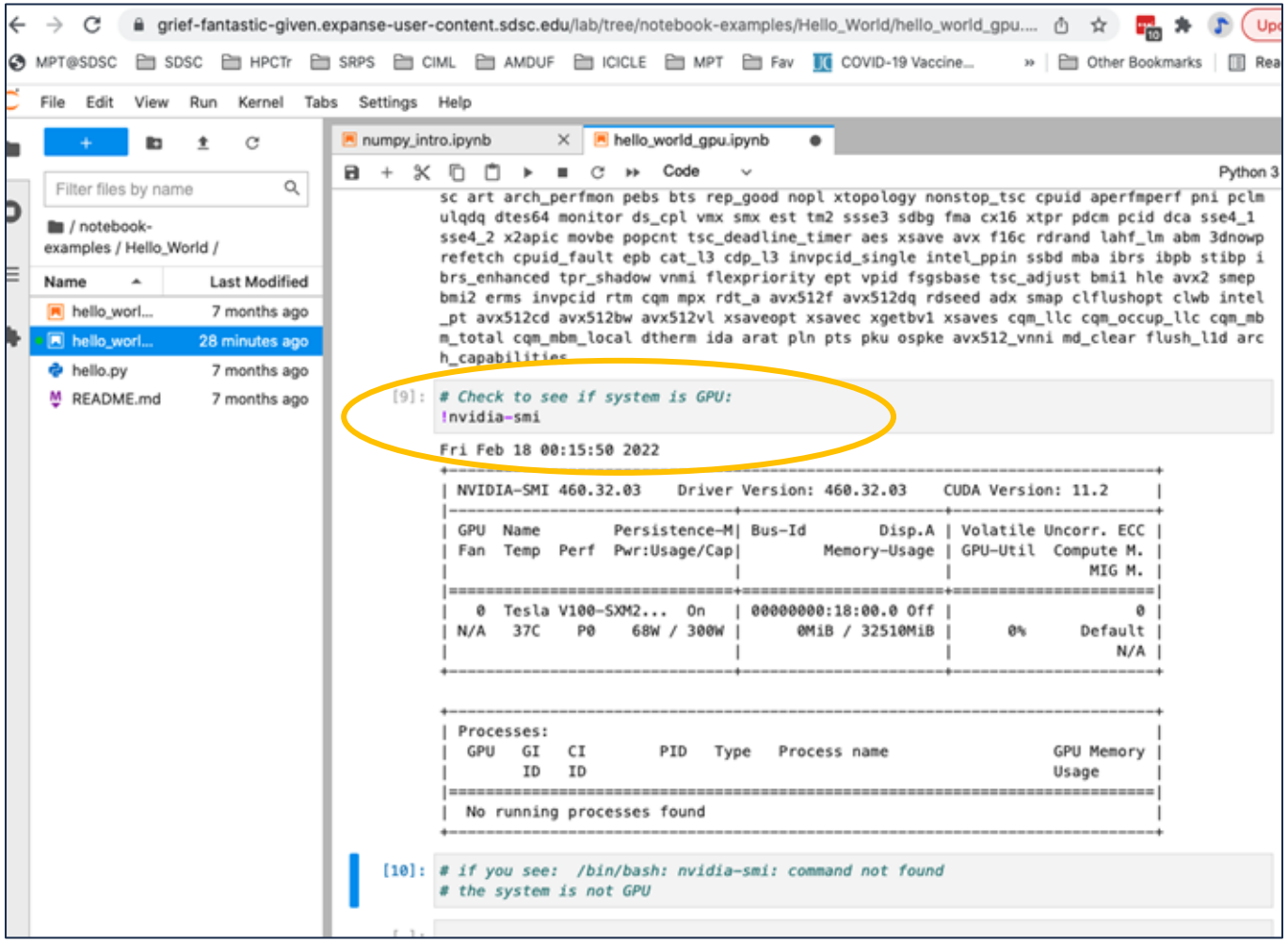




Launching GPU notebooks using galyleo

- GPU Notebooks run better when using containers. SDSC maintains several containers on Expanse
- See: /cm/shared/apps/containers/singularity

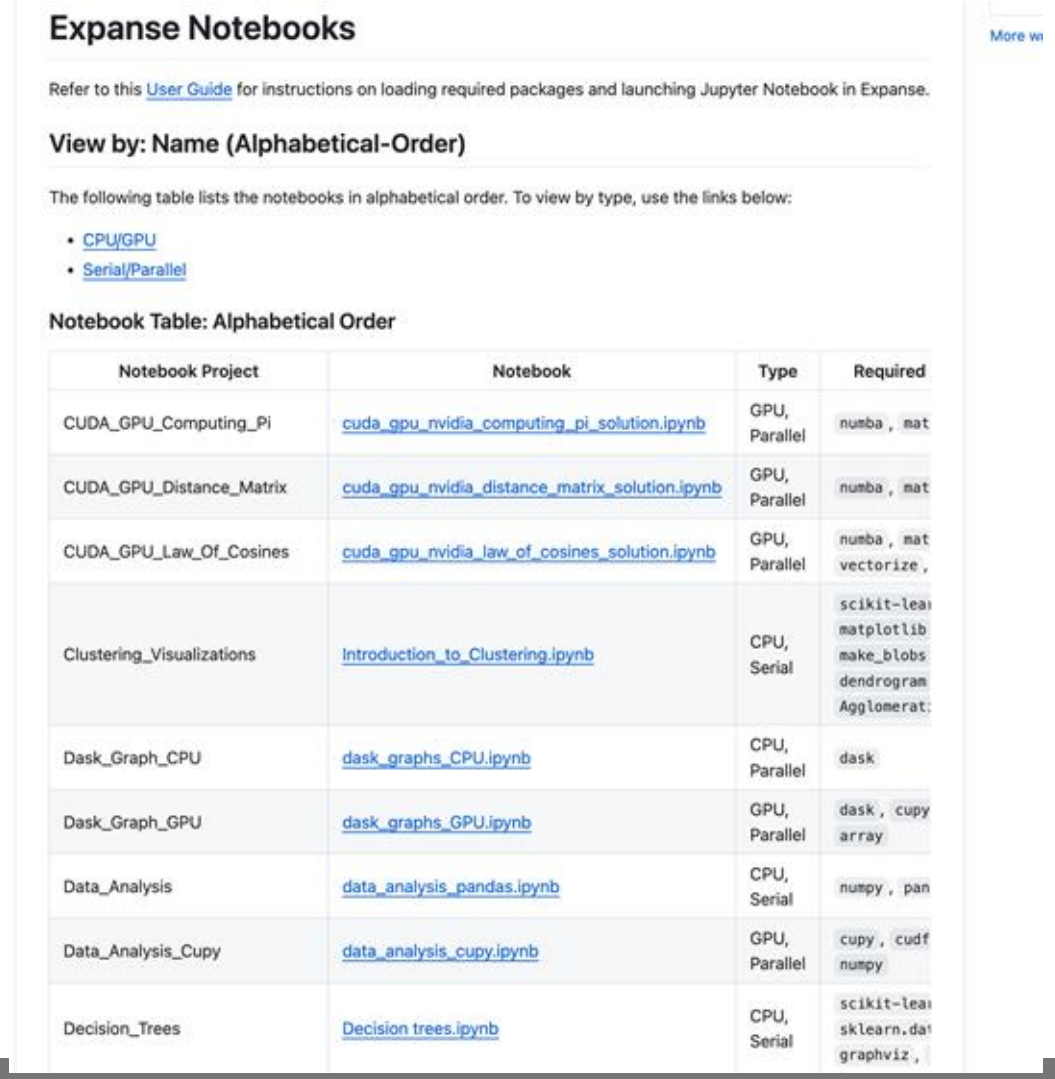
Verify notebook launched on GPU device



Notebook Examples

https://github.com/sdsc-hpc-training-org/Expanse-Notebooks

- Collection of notebooks tested on Expanse and other SDSC HPC systems
- Includes range of materials from “hello world” to ML notebooks.
- View by name, CPU/GPU, or Serial/Parallel



Expanse Notebooks

Refer to this User Guide for instructions on loading required packages and launching Jupyter Notebook in Expanse.

View by: Name (Alphabetical-Order)

The following table lists the notebooks in alphabetical order. To view by type, use the links below:

- CPU/GPU
- Serial/Parallel

Notebook Table: Alphabetical Order

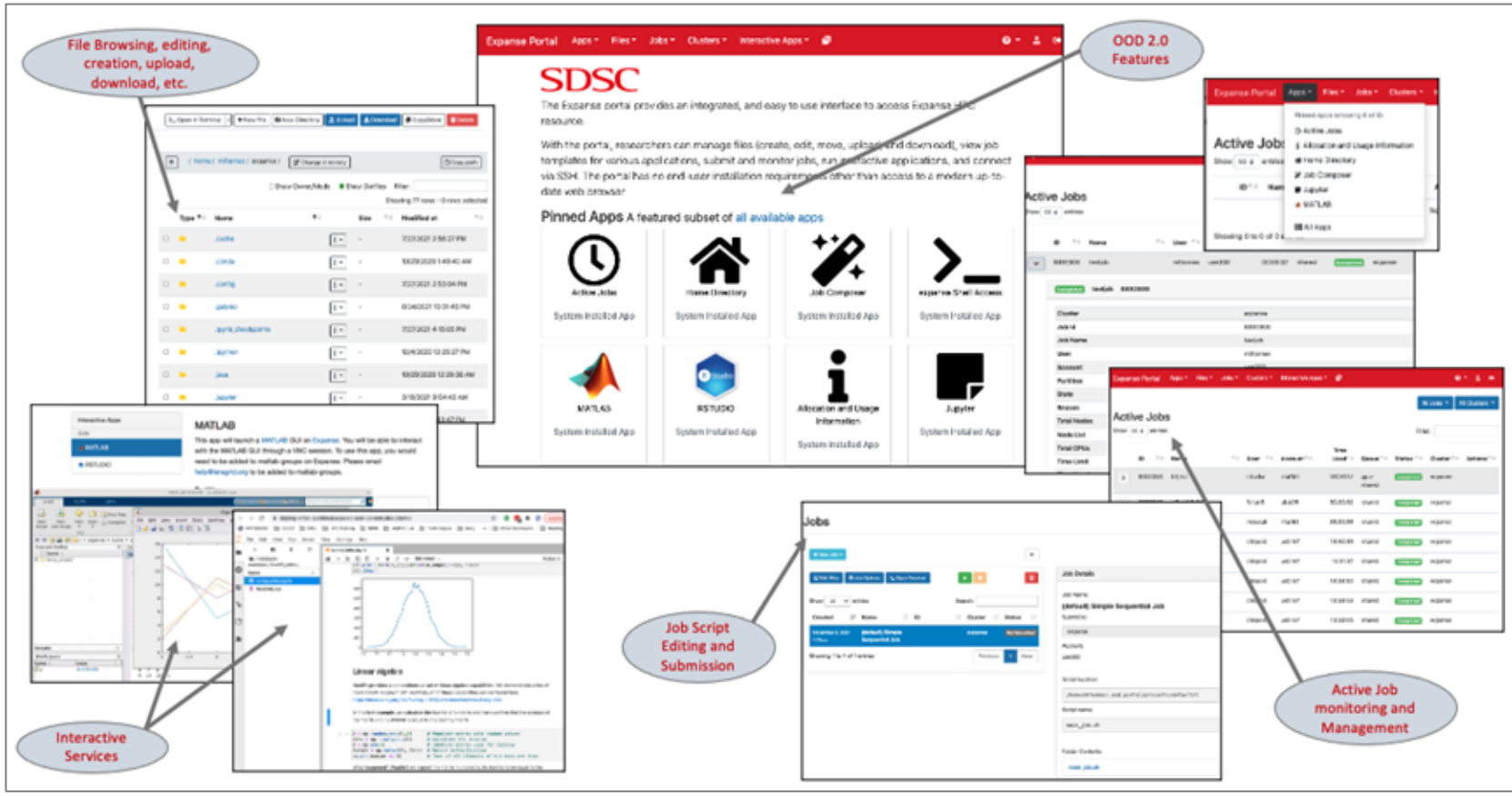
Notebook Project	Notebook	Type	Required
CUDA_GPU_Computing_Pi	cuda_gpu_nvidia_computing_pi_solution.ipynb	GPU, Parallel	numba , mat
CUDA_GPU_Distance_Matrix	cuda_gpu_nvidia_distance_matrix_solution.ipynb	GPU, Parallel	numba , mat
CUDA_GPU_Law_Of_Cosines	cuda_gpu_nvidia_law_of_cosines_solution.ipynb	GPU, Parallel	numba , mat vectorize ,
Clustering_Visualizations	Introduction_to_Clustering.ipynb	CPU, Serial	scikit-lea matplotlib make_blobs dendrogram Agglomerat
Dask_Graph_CPU	dask_graphs_CPU.ipynb	CPU, Parallel	dask
Dask_Graph_GPU	dask_graphs_GPU.ipynb	GPU, Parallel	dask , cupy array
Data_Analysis	data_analysis_pandas.ipynb	CPU, Serial	numpy , pan
Data_Analysis_Cupy	data_analysis_cupy.ipynb	GPU, Parallel	cupy , cudf numpy
Decision_Trees	Decision trees.ipynb	CPU, Serial	scikit-lea sklearn.da graphviz ,

Table of Contents: Part 2

SIMPLIFY! USE THE EXPANSE PORTAL

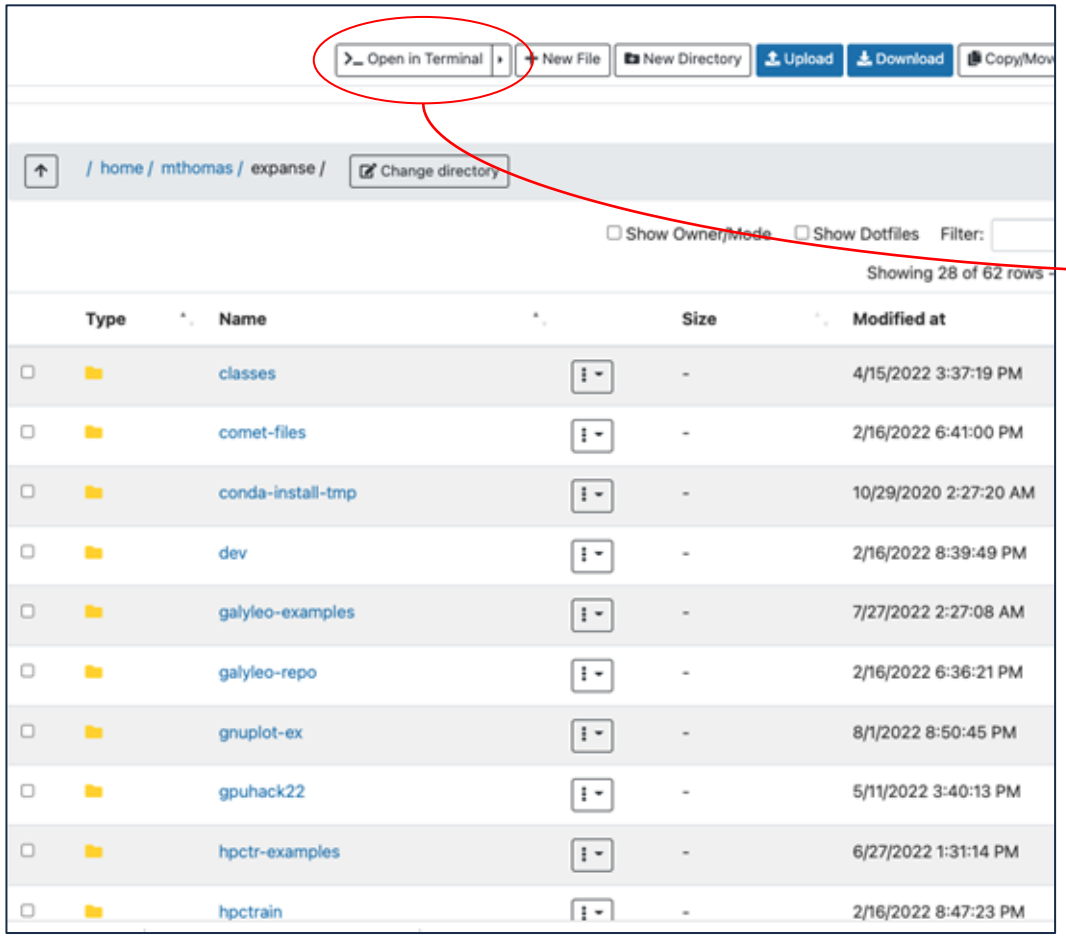
Expanse User Portal

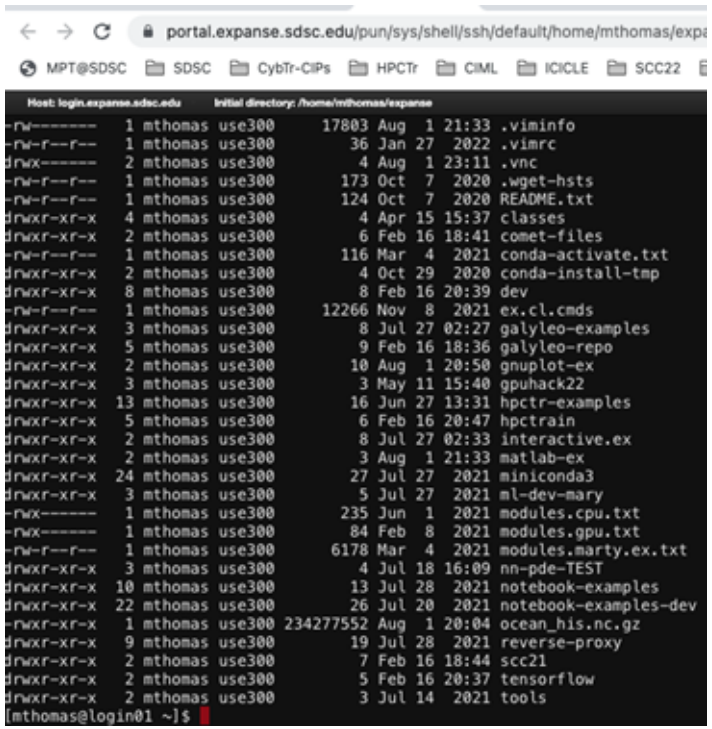
- https://portal.expanse.sdsc.edu
- authenticate using ACCESS credentials
- Securely hosts batch job submission & monitoring, and interactive applications
- Simplifies launching supported interactive applications
- manages software dependencies
- Based on OOD portal



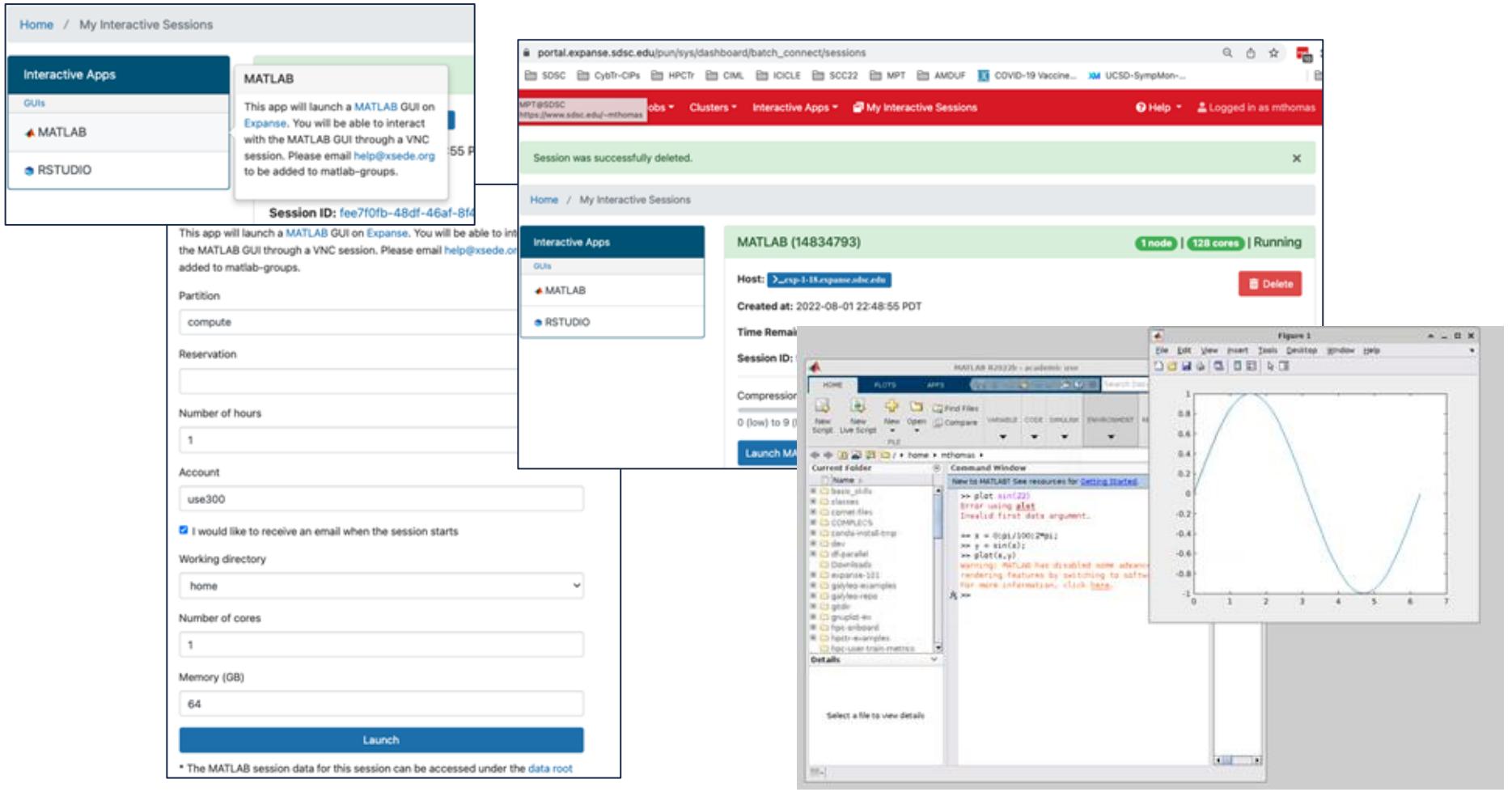
Sivagnanam, S., Irving, O., Randles, M., Mishin, D., Sakai, S., Strande, S., Yatheni, M., Thomas, M., Norman, M. (2021). Experiences in building a user portal for Expanse supercomputer. *PEARC21: Practice and Experience in Advanced Research Computing*, Accepted. ACM. https://doi.org/10.1145/3437359.3465590

Expanse Portal: File Management

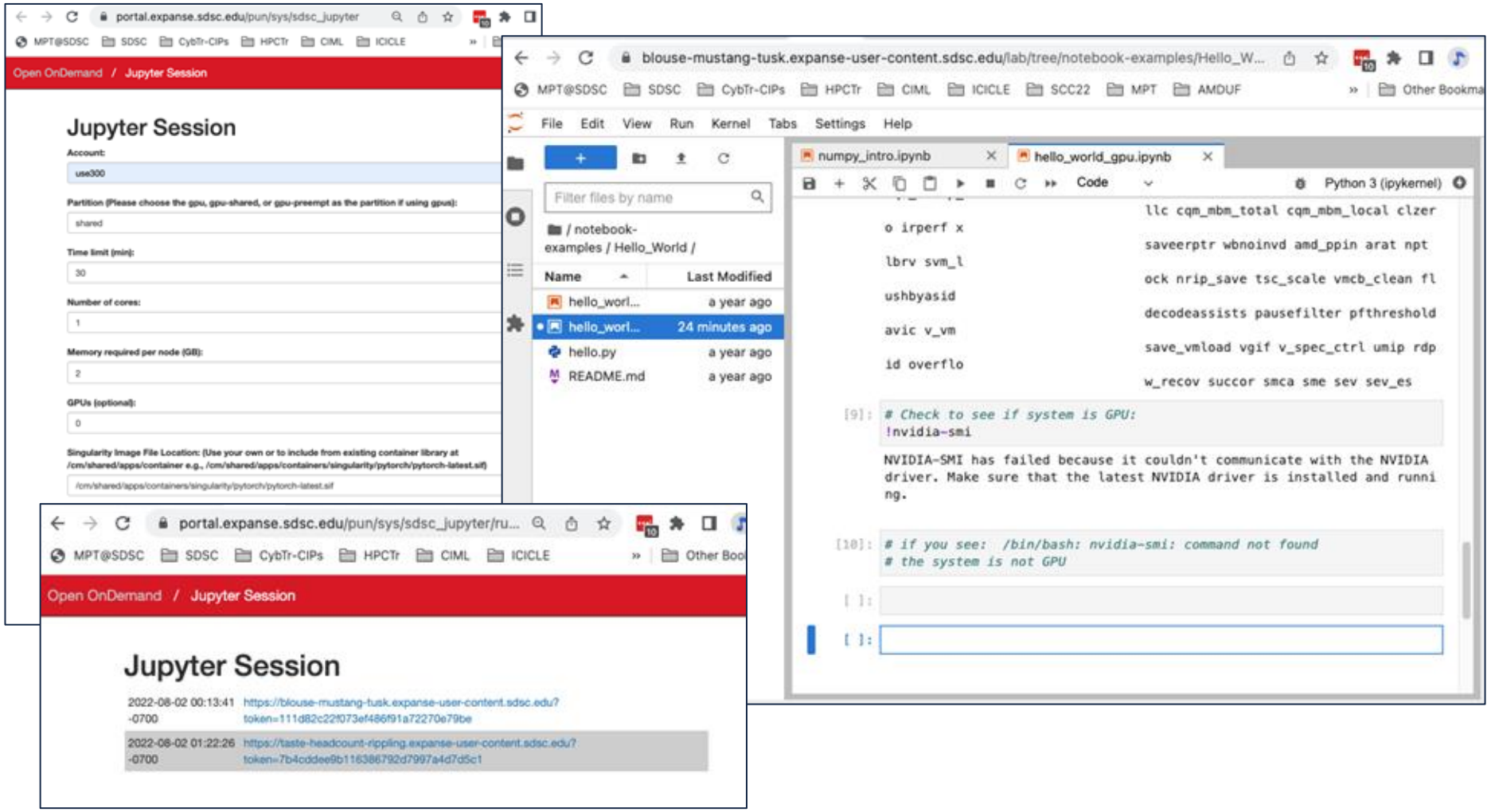




Expanse Portal: Running Matlab



Expanse Portal: Launching Notebooks



Expanse Portal: Input data example for Jupyter Notebook

Account: <abc123>
Partition: *(Please choose the gpu, gpu-shared, or gpu-preempt as the partition if using gpus): debug*
Time limit (min): 30
Number of cores: 1
Memory required per node (GB): 2
GPUs (optional): 0
Singularity Image File Location: *(Use your own or to include from existing container library at /cm/shared/apps/container e.g., /cm/shared/apps/containers/singularity/pytorch/pytorch-latest.sif)*
/cm/shared/apps/containers/singularity/pytorch/pytorch-latest.sif
Environment modules to be loaded *(E.g., to use latest version of system Anaconda3 include cpu,gcc,anaconda3):* singularitypro
Conda Environment (Enter your own conda environment if any):
Conda Init (Provide path to conda initialization scripts):
Conda Yaml *(Upload a yaml file to build the conda environment at runtime) No file chosen:*
Turn on use of mamba for speeding up conda-yml installs:
Enable use of new caching mechanism that will store and reuse conda-yml created environments using conda-pack !????
Reservation: complecs
QoS:
Working directory: HOME
Type: JupyterLab

Thank You!

Q&A

If you have problems, please contact **consult@sdsc.edu**

https://github.com/sdsc-complecs/interactive-computing/

Some final notes

- When you get started on a new HPC system, or begin working with a new piece of software, you may not know the computational requirements for the types of workloads you need to run. Experiment! Start small on number of CPUs/GPUs, amount of memory, then scale up your job resource requests as needed. Start with long on max wallclock runtimes (`#SBATCH --time`), then dial it back.
- Be aware of how your HPC system charges for the resources requested from the scheduler. Some charge by node-hour, some by CPU-core-hour, some by GPU-hour, some by memory, or even a combination of all of the above. In general, you are only charged for the resources consumed, not those requested.
- Read your HPC system's user guide, the Slurm documentation, and/or the documentation to the specific software you're attempting to run.
- If you have any questions, please don't hesitate to contact your HPC user services support team. We are there to help you understand how to utilize these specialized resources effectively for your research.

Additional Information

- Slurm Quick Start User Guide
- HPCWIKI: Scheduling Basics
- Introduction to HPC
- HPC in a Day
- Google's Shell Style Guide

Scheduler Rosetta Stone

Common User Commands					
	PBS/Torque	**Slurm**	**LSF**	**SGE**	**LoadLeveler**
Job submission	qsub [script_file]	sbatch [script_file]	bsub [script_file]	qsub [script_file]	llsubmit [script_file]
Job deletion	qdel [job_id]	scancel [job_id]	bkill [job_id]	qdel [job_id]	llcancel [job_id]
Job status	qstat [job_id]	squeue [job_id]	bjobs	qstat [-j job_id]	llq -u [username]
Job status (by user)	qstat -u [user_name]	squeue -u [user_name]	bjobs	qstat [-j job_id]	llq -u [user_name]
Job hold	qhold [job_id]	scontrol hold [job_id]	bstop [job_id]	qhold [job_id]	llhold -r [job_id]
Job release	qrls [job_id]	scontrol release [job_id]	bresume [job_id]	qrls [job_id]	llhold -r [job_id]
Queue list	qstat -Q	squeue	bqueues	qconf -sql	llclass
GUI	xpbsmon	sview	xlsf/xlsbatch	N/A	xload
Environment Variables					
	PBS/Torque	**Slurm**	**LSF**	**SGE**	**LoadLeveler**
Job ID	$PBS_JOBID	$SLURM_JOBID	$LSB_JOBID	$JOB_ID	$LOAD_STEP_ID
Working Directory	$PBS_O_WORKDIR	Use "pwd" command	Use "pwd" command	$SGE_O_WORKDIR	$LOADL_STEP_INITDIR
Node List	$PBS_NODEFILE	$SLURM_JOB_NODELIST	$LSB_HOSTS/LSB_MCPU_HOST	$PE_HOSTFILE	$LOADL_PROCESSOR_LIST
Job Specification					
	PBS/Torque	**Slurm**	**LSF**	**SGE**	**LoadLeveler**
Script directive	#PBS	#SBATCH	#BSUB	#$	#@
Queue	-q [queue]	-p [queue]	-q [queue]	-q [queue]	class=[queue]
Node Count	-l nodes=[count]	-N [min[-max]]	-n [count]	N/A	node=[count]
CPU Count	-l ppn=[count]	-n [count]	-n [count]	-pe ompi [#]	tasks_per_node=[count]
Wall Clock Limit	-l walltime=[hh:mm:ss]	-t [min] OR -t [days-hh:mm:ss]	-W	-l time=[hh:mm:ss]	wall_clock_limit=[hh:mm:ss]
Standard Output File	-o [file_name]	-o [file_name]	-o [file_name]	-o [path]	output=[file]
Standard Error File	-e [file_name]	e [file_name]	-e [file_name]	-e [path]	error=[file]
Combine stdout/err	-j oe (both to stdout) OR -j eo (both to stderr)	(use -o without -e)	(use -o without -e)		
Copy Environment	-V	--export=[ALL | NONE | variables]	?	-V	environment=COPY_ALL
Event Notification	-m abe	--mail-type=[events]	-B or -N	-m abe	notification=start|error|complete|never|always
Email Address	-M [address]	--mail-user=[address]	-u [address]	-M [address]	notify_user=[address]
Job Name	-N [name]	--job-name=[name]	-J [name]	-N [name]	job_name=[name]
Job Restart	-r [y|n]	--requeue OR --no-requeue (NOTE: configurable default)	-r	-r [yes|no]	restart=[yes|no]
Job Type	N/A	N/A	N/A	N/A	job_type=[type]
Working Directory	N/A	--workdir=[dir_name]	(submission directory)	-wd [directory]	initialdir=[directory]
Resource Sharing	-l naccesspolicy=singlejob	--exclusive OR--shared	-x	N/A	node_usage=not_shared
Memory Size	l mem=[MB]	--mem=[MB] OR --mem-per-cpu=[MB]	-M [MB]	-mem [GB]	requirements=(Memory >= [MB])
Project to charge	-W group_list=<project>	--acount=[project]	-P <project>		
Resource Requirements			-R		
Job dependency		--depend=[job_id:state]	-w [done | exit | finish]		
Job project		--wckey=[name]	-P [name]		
Job host preference		--nodelist=[nodes] AND/OR --exclude=[nodes]	-m [nodes]		

https://slurm.schedmd.com/rosetta.html

Resources

- GitHub Repo for this presentation:
  - https://github.com/sdsc-complecs/interactive-computing/
- SDSC Training Resources
  - https://www.sdsc.edu/education_and_training/training
  - Code: https://github.com/sdsc-hpc-training-org/hpctr-examples
  - Running notebooks
    - using *galyleo:* https://github.com/mkandes/galyleo
    - Examples: https://github.com/sdsc-hpc-training-org/notebook-examples
- Expanse :
  - Landing page: expanse.sdsc.edu
  - User Guide: https://expanse.sdsc.edu/support/user_guides/expanse.html
  - Training: https://www.sdsc.edu/education_and_training/training_hpc.html
- Problems? Contact consult@sdsc.edu